

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2022 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютеризованих систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Програмне забезпечення для перекладу та сумаризації новин

Виконала студентка IV курсу, групи ІП-82
(шифр групи)

Рибалко Анастасія Анатоліївна
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник доцент, доц., Лебідь С.О.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент ас. каф. ОТ Нечай Д.О.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютеризованих систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ____ ” _____ 2022 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Рибалко Анастасії Анатоліївни

(прізвище, ім'я, по батькові)

1. Тема проєкту Програмне забезпечення для перекладу та сумаризації новин

керівник проєкту Лебідь Сергій Олександрович, доцент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «10»червня 2022 р. №1033-с

2. Термін подання студентом проєкту « 19 » червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази даних.

3) Математичне забезпечення: змістовна та математична постановки задачі, обґрунтування та опис методу розв'язання.

4) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного забезпечення, архітектура програмного забезпечення, побудова звітів.

5) Технологічний розділ: керівництво користувача, методика випробувань програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань _____

2) Схема реалізованої архітектури трансформера _____

3) Приклад роботи програмного забезпечення _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «10» березня 2022 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	10.03.2022	
2	Аналіз існуючих методів розв'язання задачі	21.03.2022	
3	Постановка та формалізація задачі	15.04.2022	
4	Розробка інформаційного забезпечення	28.04.2022	
5	Алгоритмізація задачі	05.05.2022	
6	Обґрунтування вибору використаних технічних засобів	07.05.2022	
7	Розробка програмного забезпечення	22.05.2022	
8	Налагодження програми	30.05.2022	
9	Виконання графічних документів	02.06.2022	
10	Оформлення пояснювальної записки	07.06.2022	
11	Подання ДП на попередній захист	06.06.2022	
12	Подання ДП рецензенту	12.06.2022	
13	Подання ДП на основний захист	19.06.2022	

Студент

(підпис)

Анастасія РИБАЛКО

(ініціали, прізвище)

Керівник

(підпис)

Сергій ЛЕБІДЬ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з чотирьох розділів, містить 13 таблиць, 13 рисунків та 8 джерел – загалом 45 сторінок.

Дипломний проєкт присвячений таким задачам обробки природної мови за допомогою нейронних мереж як переклад та узагальнення тексту.

Мета: Створити програмне забезпечення для перекладу та сумаризації новин за допомогою використання нейронних мереж.

Об'єкт дослідження: програмне забезпечення для перекладу та сумаризації новин.

Предмет дослідження: процес розроблення нейронних мереж для перекладу тексту і для сумаризації тексту, процес розробки веб-застосунку

У першому розділі проведено аналіз предметної області, розглянуто існуючі рішення, визначено мету роботи та вимоги до програмного забезпечення.

У другому розділі проаналізовано вимоги і зроблено висновки щодо архітектури програмного забезпечення та методів її імплементації. Також було описано кроки побудови сервісу.

Третій розділ присвячено аналізу якості програмного забезпечення. Описано використані методики та контрольні приклади.

Четвертий розділ присвячено розгортанню програмного забезпечення та взаємодії з ним користувача.

КЛЮЧОВІ СЛОВА: ОБРОБКА ПРИРОДНОЇ МОВИ, TRANSFORMER, ATTENTION MECHANISM, FLASK.

ABSTRACT

. The explanatory note of the diploma project consists of four sections, contains 13 tables, 13 figures and 13 sources - a total of 46 pages.

The diploma project is devoted to such tasks of natural language processing with the help of neural networks as translation and summarization of the text.

Objective: To create software for translating and summarizing news using neural networks.

Object of research: software for translation and summarization of news.

Subject of research: the process of developing neural networks for text translation and text summarization, the process of developing a web application

In the first section the analysis of the subject area is carried out, the existing decisions are considered, the purpose of work and requirements to the software are defined.

The second section analyzes the requirements and draws conclusions about the software architecture and methods of its implementation. The steps of building the service were also described.

The third section is devoted to software quality analysis. The used methods and control examples are described.

The fourth section is devoted to the deployment of software and user interaction with it.

KEY WORDS: NATURAL LANGUAGE PROCESSING, TRANSFORMER, ATTENTION MECHANISM, FLASK.

**Пояснювальна записка
до дипломного проєкту**

на тему: Програмне забезпечення для перекладу та сумаризації новин

КПІ.ПІ-8228.045440.02.81

Київ – 2022

ЗМІСТ

ВСТУП	10
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	12
1.1 Загальні положення	12
1.2 Змістовний опис і аналіз предметної області	14
1.3 Аналіз успішних ІТ-проектів.....	15
1.3.1 Аналіз відомих технічних рішень.....	15
1.3.2 Аналіз відомих програмних продуктів.....	21
1.4 Аналіз вимог до програмного забезпечення	22
1.4.1 Розроблення функціональних вимог	22
1.4.2 Розроблення нефункціональних вимог	25
1.5 Постановка задачі	26
Висновки до розділу	26
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	28
2.1 Моделювання та аналіз програмного забезпечення.....	28
2.2 Архітектура програмного забезпечення.....	33
2.3 Конструювання програмного забезпечення.....	34
2.3.1 Введення в архітектуру трансформера.....	34
2.3.2 Основні функції та класи.	40
2.4 Аналіз безпеки даних	44
Висновки до розділу	44
3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ 45	
3.1 Аналіз якості ПЗ.....	45
3.2 Опис процесів тестування.....	45
Висновки до розділу	50
4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.	51

4.1	Розгортання програмного забезпечення.....	51
4.2	Робота з програмним забезпеченням.....	52
	Висновки до розділу	52
	ВИСНОВКИ.....	53
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	55

ВСТУП

В житті сучасного суспільства високу позицію займає інформація. Вона є базою для прийняття будь-яких рішень. Дослідження інформації – актуальна тема протягом довгого часу. Інформаційний простір визначає призму, через яку людина бачить світ, а думки мас мають вплив на політику держав.

Право на доступ до інформації є важливою складовою демократичного суспільства, що дозволяє громадянам притягувати своїх обраних представників до відповідальності за рішення, які вони приймають, і за те, як вони витрачають державні кошти. Це визнається основним правом провідними правозахисними органами та судами.

Тому однією з найважливіших задач, що постає перед кожним діячем, рухом або спільнотою є ефективно донесення своєї перспективи до глобальної аудиторії. А однією з найскладніших задач, що постає перед індивідом, що намагається сформулювати об'єктивний погляд на речі є аналіз та порівняння різних бачень та відділення зерен від половини. У наданні інструментів, які можуть допомогти у вирішенні зазначених проблем значну роль відіграє галузь обробки природної мови (Natural Language Processing або NLP), що експоненційно швидко розвивається в останні роки.

Неоднозначність і креативність людської мови – це лише дві характеристики, які роблять NLP вибагливою сферою.

Тому питання побудови NLP-систем є досить актуальним на даний момент. Метою даного проекту є створення програмного забезпечення з користувацьким інтерфейсом для перекладу новин з різних джерел на англійську та їх сумаризації за допомогою моделей глибокого навчання. Основною ідеєю є надання людям доступу до новинних джерел на різних мовах з різних куточків світу і можливості швидко отримувати екстракцію основної інформації з цих джерел. Це дозволить отримувати більшу обізнаність у темах, які майже не освітлюються у звичайно легко доступних

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

людині джерелах, та спростить процес аналізу різних перспектив та порівняння наративів.

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

Обробка природної мови (NLP) — це галузь, що знаходиться на перетині інформатики, штучного інтелекту та лінгвістики. Вона займається створенням систем, які можуть обробляти й розуміти людську мову. З моменту свого заснування в 1950-х роках і до недавнього часу NLP була в першу чергу сферою академічних та дослідницьких робіт, що вимагало формальної освіти та тривалого навчання. Прориви останнього десятиліття призвели до того, що NLP все ширше використовується в різних галузях, таких як роздрібна торгівля, охорона здоров'я, фінанси, право, маркетинг, людські ресурси та багато інших.

Існує набір фундаментальних завдань, які часто зустрічаються в різних проектах NLP:

- Мовне моделювання. Це завдання передбачити, яким буде наступне слово в реченні на основі історії попередніх слів. Мета цього завдання — дізнатися ймовірність появи послідовності слів у даній мові. Мовне моделювання корисно для створення рішень для широкого спектру проблем, таких як розпізнавання мовлення, оптичне розпізнавання символів, розпізнавання рукописного тексту, машинний переклад і виправлення орфографії.
- Класифікація тексту. Це завдання розділити текст на відомий набір категорій на основі його змісту. Класифікація тексту, безумовно, є найпопулярнішою задачею в NLP і використовується в різних інструментах, від ідентифікації спаму електронної пошти до аналізу настроїв.
- Вилучення інформації. Як випливає з назви, це завдання вилучення відповідної інформації з тексту, наприклад,

календарних подій з електронних листів або імен людей, згаданих у публікації в соціальних мережах.

- Пошук інформації. Це завдання пошуку документів, що відповідають запиту користувача, із великої колекції. Такі програми, як Пошук Google, добре відомі для пошуку інформації.
- Розмовні агенти. Це завдання побудови діалогових систем, які можуть спілкуватися людськими мовами. Alexa, Siri тощо є деякими поширеними додатками цього завдання.
- Узагальнення (сумаризація) тексту. Це завдання спрямоване на створення коротких резюме довших документів, зберігаючи основний зміст тексту.
- Відповіді на запитання. Це завдання побудови системи, яка може автоматично відповідати на запитання, поставлені природною мовою.
- Машинний переклад. Це завдання перетворення фрагмента тексту з однієї мови на іншу. Такі інструменти, як Google Translate, є поширеними додатками для цього завдання.
- Тематичне моделювання. Це завдання розкриття актуальної структури великої колекції документів. Тематичне моделювання є звичайним інструментом для вивчення тексту і використовується в широкому діапазоні областей, від літератури до біоінформатики.

Дана робота присвячена таким задачам обробки природної мови як машинний переклад тексту та сумаризація тексту.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		13

1.2 Змістовний опис і аналіз предметної області

Переклад та сумаризація тексту являють собою галузі, що розвиваються з експоненційно зростаючою швидкістю останнє десятиліття. На це існує ряд причин:

- Широко доступні та прості у використанні інструменти, техніки та API NLP набувають поширення
- Розробка більш доступних інтерпретації і узагальнених підходів покращила базову продуктивність навіть для складних завдань NLP
- Все більше і більше організацій, включаючи Google, Microsoft і Amazon, вкладають значні кошти в більш інтерактивні споживчі продукти, де мова використовується як основний засіб спілкування.
- Збільшення доступності корисних наборів даних із відкритим вихідним кодом
- Створення та поширення наборів даних і моделей для низькоресурсних мов

Дана робота присвячена створенню інструмента, що надасть швидкий доступ до інформаційних джерел з різних куточків світу та дозволить отримувати сумаризацію новин на задану тематику на англійській мові за допомогою використання нейронних мереж.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		14

1.3 Аналіз успішних ІТ-проектів

1.3.1 Аналіз відомих технічних рішень

За останні кілька років ми спостерігаємо величезний сплеск використання нейронних мереж для роботи зі складними неструктурованими даними. Мова за своєю суттю складна і неструктурована. Тому нам потрібні моделі з кращим представленням і здатністю до навчання, щоб розуміти та розв'язувати мовні завдання. Ось кілька популярних архітектур глибоких нейронних мереж, які стали статус-кво в NLP.

Рекурентні нейронні мережі (RNN)

Мова за своєю суттю послідовна. Речення будь-якою мовою перетікає з одного напрямку в інший. Таким чином, модель, яка може поступово читати введений текст від одного кінця до іншого, може бути дуже корисною для розуміння мови. Рекурентні нейронні мережі (RNN) спеціально розроблені, щоб мати на увазі таку послідовну обробку та навчання. RNN мають нейронні блоки, які здатні запам'ятовувати те, що вони обробили до цього часу. Ця пам'ять є тимчасовою, і інформація зберігається та оновлюється з кожним кроком часу, коли RNN читає наступне слово у вхідних даних.

RNN є потужними і дуже добре працюють для вирішення різноманітних завдань NLP, таких як класифікація тексту, розпізнавання іменованих об'єктів, машинний переклад тощо. Можна також використовувати RNN для створення тексту, мета якого полягає в тому, щоб прочитати попередній текст і передбачити наступне слово або наступний символ.

Рекурентна нейронна мережа (RNN) — це особливий тип штучної нейронної мережі, пристосованої для роботи з даними часових рядів або даними, які включають послідовності. Звичайні нейронні мережі прямого зв'язку призначені лише для точок даних, які не залежать одна від одної. Однак, якщо у нас є дані в такій послідовності, що одна точка даних залежить від попередньої точки даних, нам потрібно змінити нейронну мережу, щоб

включити залежності між цими точками даних. RNN мають концепцію «пам'яті», яка допомагає їм зберігати стани або інформацію попередніх входів для створення наступного виходу послідовності.

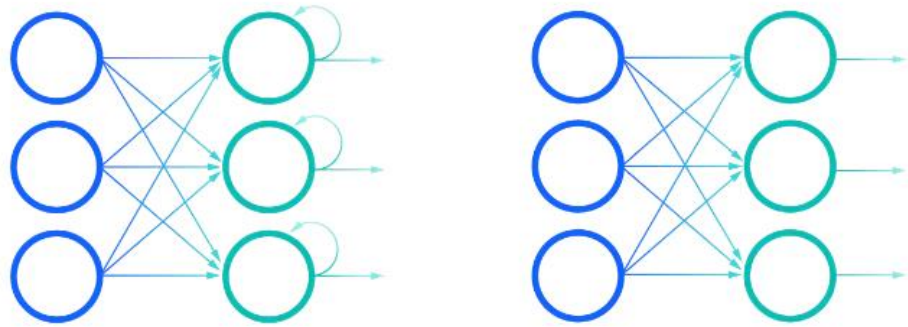


Рис. 1.1 Порівняння рекурентної нейромережі з неймережою прямого зв'язку

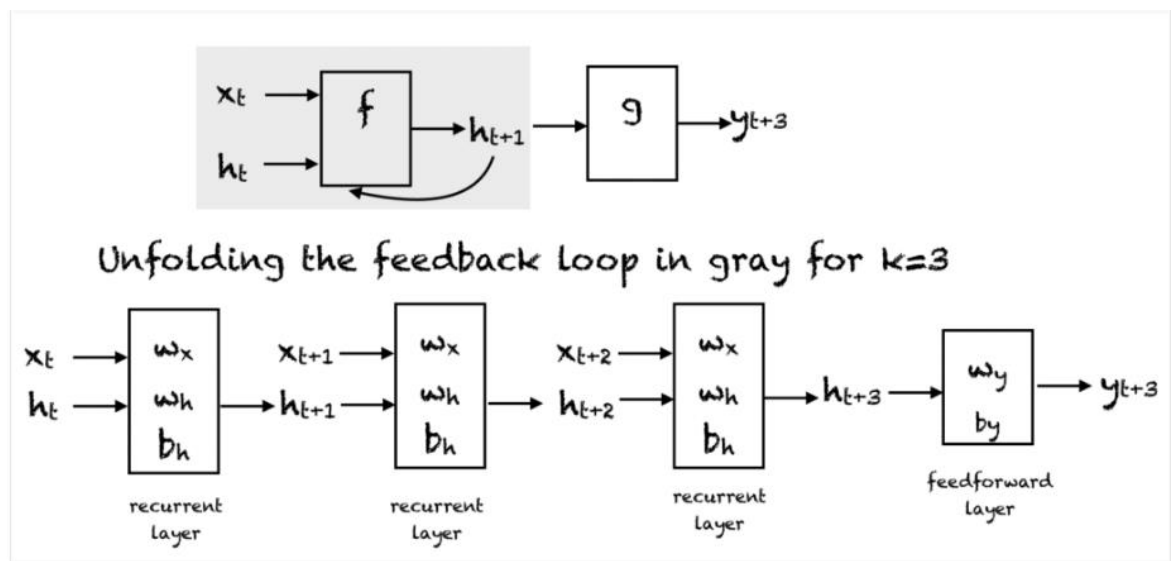


Рис. 1.2 Розгортка рекурентної нейромережі

Проста RNN має петлю зворотного зв'язку, як показано на першій діаграмі на малюнку вище. Петлю зворотного зв'язку, показану в сірому прямокутнику, можна розгорнути на 3 кроки часу, щоб створити другу мережу на малюнку вище. На малюнку використано такі позначення:

x_t — введення на етапі часу. Щоб все було просто, ми припустимо, що це скалярне значення з однією ознакою.

y_t — вихід мережі на етапі часу. Ми можемо створити кілька виходів у мережі, але для цього прикладу ми припускаємо, що є один вихід.

h_t — вектор зберігає значення прихованих одиниць/станів у момент часу. Це також називається поточним контекстом.

m — кількість прихованих одиниць, вектор ініціалізується нулем.

w_x є вагами, пов'язаними з входами в повторюваному шарі

w_h — це ваги, пов'язані з прихованими одиницями в повторюваному шарі

w_y — це ваги, пов'язані з прихованими до вихідних одиниць

b_h є зміщенням, пов'язаним з повторюваним шаром

b_y є зміщенням, пов'язаним із шаром прямого зв'язку

На кожному кроці ми можемо розгорнути мережу за k часовими кроками, щоб отримати вихід за кроком часу $k+1$. Розгорнута мережа дуже схожа на нейронну мережу з прямим зв'язком. Прямокутник у розгорнутій мережі показує операцію, що виконується.

Під час прямого проходження RNN мережа обчислює значення прихованих одиниць і вихідний результат після k кроків часу. Вагові показники, пов'язані з мережею, розподіляються тимчасово. Кожен повторюваний шар має два набори ваг: один для входу, а другий — для прихованого блоку. Останній рівень прямого зв'язку, який обчислює кінцевий результат для k -го кроку часу, схожий на звичайний рівень традиційної мережі прямого зв'язку.

LSTM

Незважаючи на свої можливості та універсальність, RNN страждають від проблеми пам'яті — вони не можуть запам'ятовувати довший контекст і, отже, погано працюють, коли введений текст довгий, що зустрічається у задачах обробки природної мови досить часто. Мережі довгострокової пам'яті (LSTM), тип RNN, були винайдені, щоб пом'якшити цей недолік RNN. LSTM обходять цю проблему, відпускаючи невідповідний контекст і запам'ятовуючи лише ту частину контексту, яка потрібна для вирішення

поставленої задачі. Це знімає навантаження на запам'ятовування дуже довгого контексту в одному векторному представленні. Завдяки цьому обхідному шляху LSTM замінили RNN у більшості програм. Інший варіант RNN, який використовується здебільшого при генерації мов, є GRU (Gated Recurrent Units).

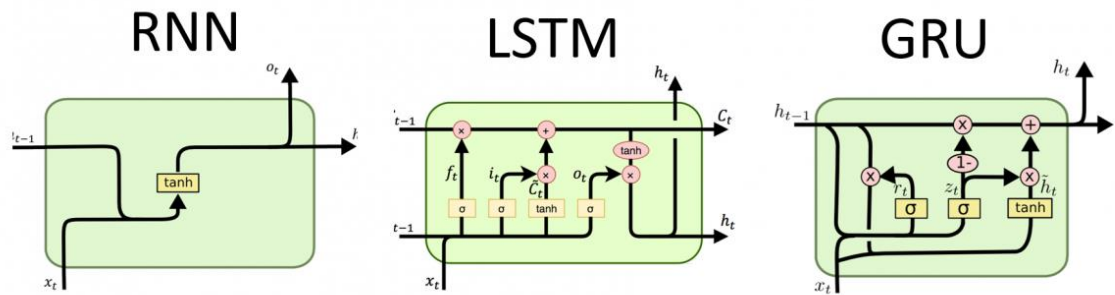


Рис 1.3 Порівняння клітин RNN, LSTM та GRU

Мережі довгострокової пам'яті, які зазвичай називають просто LSTM – це особливий тип RNN, здатний вивчати довгострокові залежності. Вони були представлені у 1997 році і були вдосконалені та популяризовані багатьма людьми в наступних роботах. Вони надзвичайно добре працюють для широкого спектру проблем і зараз багато використовуються.

LSTM розроблені, щоб уникнути проблеми довгострокової залежності. Запам'ятовування інформації протягом тривалого періоду часу – це практично їхня поведінка за замовчуванням, а не те, що їм важко вивчити.

Усі рекурентні нейронні мережі мають вигляд ланцюга повторюваних модулів нейронної мережі. У стандартних RNN цей повторюваний модуль матиме дуже просту структуру, наприклад, один шар $\tanh$.

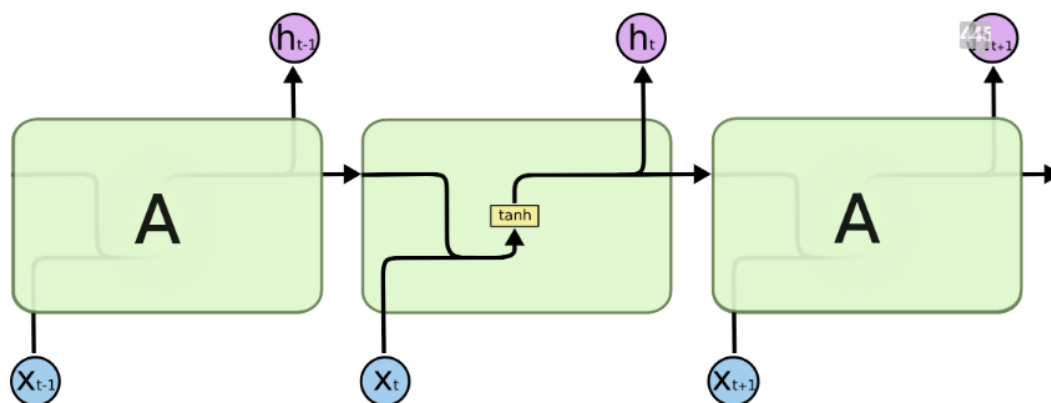


Рис 1.4 Повторюваний модуль RNN

LSTM також мають таку ланцюжкову структуру, але повторюваний модуль виглядає інакше. Замість того, щоб мати один шар нейронної мережі, є чотири, які взаємодіють дуже особливим чином.

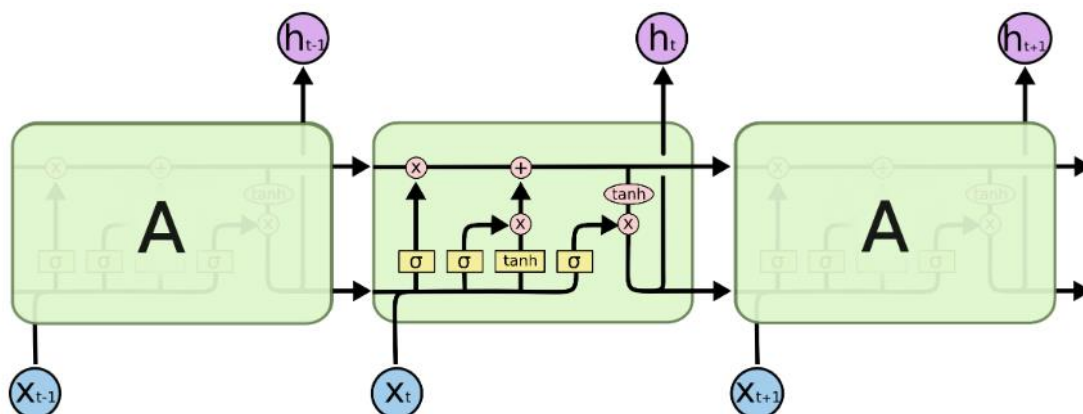


Рис 1.5 Повторюваний модуль LSTM

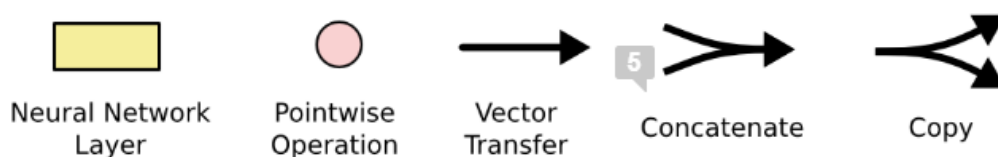


Рис 1.6 Умовні позначення

У наведеній вище діаграмі кожен рядок несе цілий вектор від виходу одного вузла до входу інших. Рожеві кола представляють точкові операції, як-от додавання векторів, а жовті квадрати — це вивчені шари нейронної мережі. Злиття рядків позначає конкатенацію, а розгалуження рядків означає, що його вміст копіюється, а копії спрямовуються в різні місця.

Головною ідеєю, що стоїть за LSTM є стан комірки, горизонтальна лінія, що проходить через верхню частину діаграми.

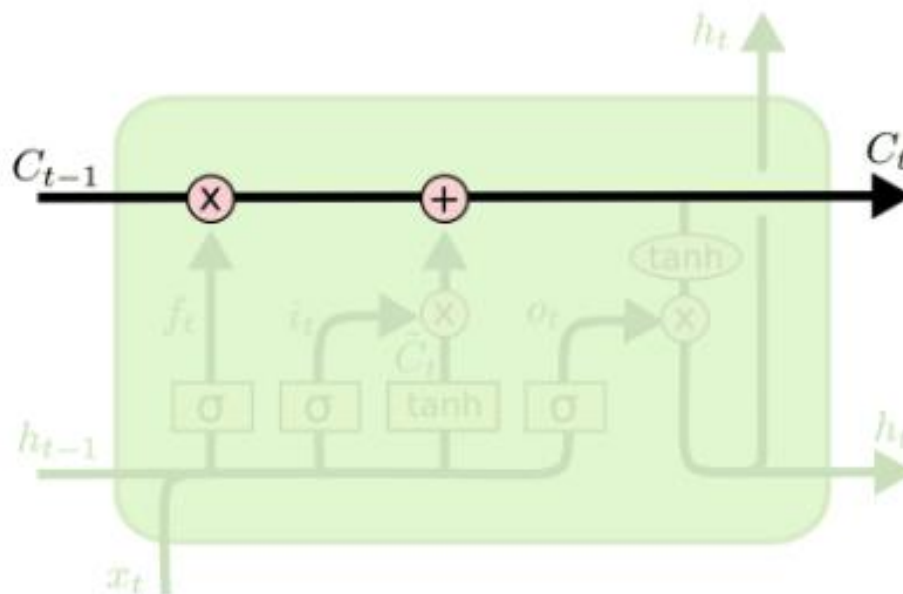


Рис 1.7 Стан комірки LSTM

LSTM має можливість видаляти або додавати інформацію до стану комірки, ретельно регульованого структурами, які називаються воротами.

Ворота – це спосіб опціонального пропускання інформації. Вони складаються із сигмоїдного шару нейронної мережі та операції поточкового множення. Сигмоїдний шар має на виході числа від нуля до одиниці, описуючи, скільки процентів кожного компонента слід пропустити. Нульове значення означає «нічого не пропускати», а значення одиниці означає «пропустити все». LSTM має три таких ворота для захисту та контролю стану клітини.

Згорткові нейронні мережі (CNN)

Згорткові нейронні мережі (CNN) дуже популярні і широко використовуються в задачах комп'ютерного зору, таких як класифікація зображень, розпізнавання відео тощо. CNN також досягли успіху в NLP, особливо в задачах класифікації тексту. Кожне слово в реченні можна замінити відповідним вектором слів, і всі вектори мають однаковий розмір. Таким чином, їх можна покласти один на одного, щоб утворити матрицю або

двовимірний масив розмірності $n \times d$, де n — кількість слів у реченні, а d — розмір векторів слів. Ця матриця тепер може розглядатися як зображення і може бути змодельована CNN. Основною перевагою CNN є їхня здатність розглядати групу слів разом за допомогою контекстного вікна. Наприклад, ми робимо класифікацію настроїв, і отримуємо речення на кшталт: «Мені дуже подобається цей фільм!» Щоб зрозуміти це речення, краще подивитись на слова та різні набори суміжних слів. CNN можуть робити саме це за визначенням своєї архітектури.

Трансформери

Трансформери є останнім продуктом серед моделей глибокого навчання для NLP. За останні два роки моделі трансформаторів досягли сучасного рівня практично у всіх основних завданнях NLP. Вони моделюють текстовий контекст, але не послідовно. Якщо ввести слово, воно вважає за краще дивитися на всі слова навколо нього (самоувага) і представляти кожне слово відповідно до його контексту. Трансформери можуть моделювати контекст і, отже, активно використовуються в задачах NLP завдяки цій вищій здатності представлення в порівнянні з іншими глибокими мережами. Ми розглянемо трансформери більш детально у наступному розділі, так як у даному проекті буде використовуватись саме ця архітектура.

1.3.2 Аналіз відомих програмних продуктів

Програмних продуктів, що реалізують окремі частини функціоналу досить багато. Для перекладу тексту існують такі проекти як Google Translate, DeepL Translator, Bing Microsoft Translator, SYSTRAN Translate, Amazon Translate, що не потребують представлення. Для сумаризації теж існує достатня кількість інструментів (менш відомих, ніж вищезазначені, але це пов'язано у першу чергу з тим, що задача сумаризації тексту рідше постає перед пересічним користувачем), наприклад, Intellippt.

1.4 Аналіз вимог до програмного забезпечення

1.4.1 Розроблення функціональних вимог

На наступних таблицях відображено сценарії взаємодії користувача з системою та функціональні вимоги, яким вона повинна відповідати.

Таблиця 1.1 UC01

Ідентифікатор	UC01
Назва	Аутентифікація
Актор	Користувач
Опис	Користувач вводить логін та пароль
Попередні умови	Користувач знає зареєстровані аутентифікаційні дані
Вихідні умови	Користувач отримує доступ до роботи з сервісом

Таблиця 1.2 UC02

Ідентифікатор	UC02
Назва	Вибір новинного джерела
Актор	Користувач
Опис	Користувач вводить адресу сайту джерела інформації, з яким хоче взаємодіяти
Попередні умови	Користувач увійшов до системи. Користувач має адресу новинного сайту
Вихідні умови	Адреса сайту збережена для подальшого скрейпінгу

Таблиця 1.3 UC03

Ідентифікатор	UC03
Назва	Вибір мови сайту
Актор	Користувач
Опис	Користувач обирає мову інформаційного джерела, з якого хоче зробити екстракцію інформації
Попередні умови	Користувач увійшов до системи. Користувач знає мову публікацій обраного джерела інформації
Вихідні умови	Обрана мова зберігається як параметр для подальшого перекладу

Таблиця 1.4 UC04

Ідентифікатор	UC04
Назва	Вибір тематики новин
Актор	Користувач
Опис	Користувач вводить у відведене поле ключове слово/слова, за якими хоче розшукувати новини
Попередні умови	Користувач увійшов до системи
Вихідні умови	Користувач отримує доступ до роботи з сервісом

Таблиця 1.5 UC05

Ідентифікатор	UC05
Назва	Переклад тексту новин
Актор	Користувач
Опис	Користувач тисне на кнопку перекладу
Попередні умови	Користувач увійшов до системи. Користувач обрав новинне джерело, його мову та тематику, що його цікавить (останнє не є обов'язковим)
Вихідні умови	Користувач отримує переклад тексту з обраного джерела на обрану тематику на англійську

Таблиця 1.6 UC06

Ідентифікатор	UC06
Назва	Переклад і сумаризація тексту новин
Актор	Користувач
Опис	Користувач тисне на кнопку сумаризації
Попередні умови	Користувач увійшов до системи. Користувач обрав новинне джерело, його мову та тематику, що його цікавить (останнє не є обов'язковим)
Вихідні умови	Користувач отримує сумаризацію тексту з обраного джерела на обрану тематику на англійській

1.4.2 Розроблення нефункціональних вимог

Сервіс повинен бути зручним у використанні і мати інтуїтивно зрозумілий інтерфейс, ознайомлення з яким не буде займати у користувача багато часу. Також програмне забезпечення повинно передбачати всі можливі варіанти взаємодії з ним та мати коректну обробку помилок.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		25

1.5 Постановка задачі

Метою розробки даного програмного забезпечення є створення сервісу для зручного доступу до новинних джерел і екстракції їх змісту мовою користувача за допомогою нейронних мереж для перекладу та сумаризації тексту.

Сервіс повинен брати на вхід адресу джерела інформації, його мову та тематику, що цікавить користувача, де адреса на джерело інформації — це посилання на новинний сайт, мова обирається з drop-down списку, а тематика може бути довільною строкою. У разі вводу некоректних даних користувач отримує повідомлення про помилку. Сервіс повинен бути зручним та мати інтуїтивно зрозумілу навігацію.

Задачами розробки даного програмного забезпечення є наступні:

- Авторизація
- Обирання новинного джерела
- Обирання мови новинного джерела
- Обирання тематики новин для здійснення фільтрації
- Здійснення фільтрації новинних статей за заданими параметрами
- Здійснення перекладу новин
- Здійснення сумаризації перекладених новин

Висновки до розділу

Сьогодні питання глобального доступу до інформації є як ніколи актуальним, тому створення сервісу, що дозволить отримувати короткий зміст новин з різних джерел мовою користувача може бути досить корисним. Існує досить багато відомих рішень, що дозволяють виконувати переклад або сумаризацію тексту, але немає відомих інструментів, заточених під взаємодію з новинними джерелами та поєднуючих зазначені функції. На даний момент домінуючою архітектурою нейронних мереж для перекладу та сумаризації

тексту є трансформери, представлені у 2017 році у статті “Attention is all you need”. Тому метою даної роботи поставлено створення сервісу для зручного доступу до новинних джерел і екстракції їх змісту англійською мовою за допомогою нейронних мереж для перекладу та сумаризації тексту.

					КПІ.ІП-8228.045440.02.81	Арк.
						27
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Як було зазначено раніше, на даний момент використання нейронних мереж є одним з найефективніших підходів до розв'язання більшості задач обробки природної мови. У світі нейронних мереж для NLP у свою чергу найпопулярнішою архітектурою є трансформери, тому у даному проекті вирішено використовувати саме їх. У зв'язку з обмеженнями, які накладає наявне для вирішення поставлених задач апаратне забезпечення, модель для сумаризації тексту буде побудовано з нуля, для перекладу з української буде дотреновано модель на нових даних, а для перекладу з інших мов використано існуючі моделі.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		28

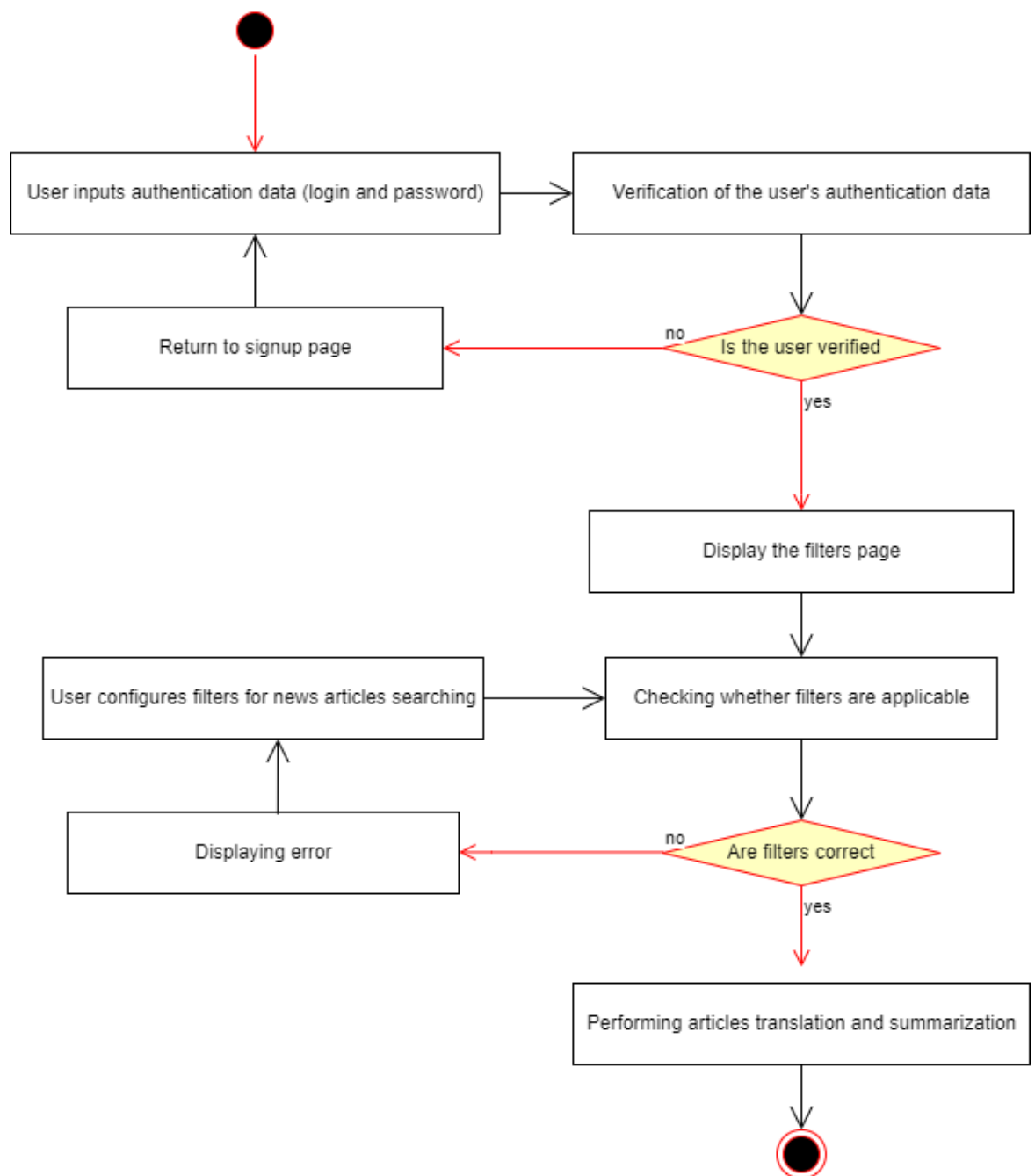


Рис. 2.1 Опис бізнес-процесів

Для реалізації цього проекту було обрано мову програмування Python.

Для побудови сумаризаційної нейронної мережі було обрано фреймворк Tensorflow, відкриту програмну бібліотека для машинного навчання, розроблену компанією Google. TensorFlow — це фреймворк машинного навчання з відкритим вихідним кодом для виконання високопродуктивних чисельних обчислень. Він дозволяє легко розгортати обчислення на

різноманітних платформах, починаючи від настільних комп'ютерів і закінчуючи кластерами серверів. Цей фреймворк було обрано за наступними причинами:

- TensorFlow забезпечує доступний і читабельний синтаксис
- TensorFlow забезпечує чудові функціональні можливості та послуги в порівнянні з іншими популярними фреймворками глибокого навчання. Ці високорівневі операції необхідні для виконання складних паралельних обчислень і для побудови передових моделей нейронних мереж.
- TensorFlow — це низькорівнева бібліотека, яка забезпечує більшу гнучкість. Вона надає можливість визначити власні функції або утиліти для моделей. Це дозволяє змінювати модель відповідно до змін вимог користувачів
- TensorFlow забезпечує більший контроль мережі, дозволяє відстежувати нові зміни, внесені з часом

Для перекладу вирішено використовувати інструменти, що пропонує HuggingFace та каталог моделей-трансформерів, що пропонує HuggingFace Hub. Це пов'язано з тим, що застосування нової архітектури машинного навчання до нового завдання зазвичай включає в себе наступні кроки: реалізація архітектури моделі в коді, як правило, на основі PyTorch або TensorFlow, завантаження попередньо підготовлених ваг (якщо доступні) із сервера, попередня обробка вхідних даних, пропуск їх через модель і застосування постобробки для конкретного завдання, визначення функцій втрат і оптимізаторів для навчання моделі. Кожен із цих кроків вимагає спеціальної логіки для кожної моделі та завдання. Традиційно, коли дослідницькі групи публікують нову статтю, вони також випускають код разом із ваговими показниками моделі. Однак цей код рідко стандартизований і часто вимагає кількох днів інженерії, щоб адаптуватися до нових випадків використання. Це те, з чим HuggingFace допомагає розробникам у сфері NLP.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		30

Він надає стандартизований інтерфейс для широкого спектру моделей трансформерів, а також код і інструменти для адаптації цих моделей до нових випадків використання. На даний момент бібліотека підтримує три основні фреймворки глибокого навчання (PyTorch, TensorFlow і JAX) і дозволяє легко перемикатися між ними. Крім того, він надає заготовки для конкретних завдань, таких як класифікація тексту, розпізнавання іменованих об'єктів та відповіді на запитання. Це значно скорочує час, необхідний для навчання та тестування моделей.

Для більшої частини скрейпінгу новинних джерел вирішено скористатися інструментарієм, що надає бібліотека Newspaper3k.

Для розгортання сервісу вирішено використовувати мікрофреймворк для вебдодатків Flask. На нього вибір упав за наступними причинами:

- Масштабованість. У сучасному швидкоплинному середовищі більшість людей починають з ідеї невеликої веб-програми, а потім масштабують її відповідно до вимог. Flask може обробляти зростаючу з розширенням сервісу кількість запитів. Наприклад, Pinterest перейшов з Django на Flask і обробляє мільярди запитів на день. Причиною масштабованої природи Flask є його здатність модулювати кодову базу в міру її зростання протягом певного періоду.
- Простота у використанні та гнучкість. Є дуже мало частин Flask, які не можна замінити чи змінити. У порівнянні з Django, Flask відкритий для змін і додає рівень гнучкості в процес розробки веб-додатків. Крім того, фреймворк простий у використанні та розумінні навіть для новачка. Основною причиною простоти Flask є зрозуміла документація.
- Контроль над кодом. Flask — один з найвідоміших фреймворків для веб-додатків, який дозволяє розробникам мати повний контроль над кодовою базою. Фреймворк дозволяє розробникам вибирати компоненти, які вони хочуть для конкретної програми.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		31

- Підтримка тестування. Після розробки наступним етапом життєвого циклу програмного забезпечення є тестування. Flask має вбудовану систему модульного тестування, яка дає змогу легко протестувати програму, перш ніж запустити її у продакшн. Крім того, фреймворк надає такі функції, як вбудований сервер розробки та швидкий налагоджувач. Це допомагає зробити процес тестування безпроблемним. Крім того, фреймворк надзвичайно легкий.
- Спрощене експериментування. Flask дозволяє адаптуватися до нових технологій і працювати за допомогою гнучких методологій розробки. Також він дозволяє легко додавати новий функціонал до продукту з його популяризацією за допомогою систем швидкої інтеграції та підтримки.

Для реалізації функціоналу, пов'язаного безпосередньо з взаємодією з моделями, було обрано фреймворк для розгортання демо моделей машинного навчання Gradio. Gradio дозволяє створювати демонстрації роботи моделей машинного навчання та ділитися ними, використовуючи лише Python. Також Gradio підтримує інтеграцію з HuggingFace. Для хостингу буде використовуватись платформа AWS.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		32

2.2 Архітектура програмного забезпечення

Можна виділити чотири логічно відокремлені модулі:

- Побудова та тренування нейронної мережі для сумаризації тексту. Задачою цього модулю є створення архітектури нейронної мережі-трансформера для сумаризації тексту та її тренування. Після цього ми отримаємо вже натреновану модель, що буде вбудована у сервіс і використовуватиметься для екстракції головних тез з довгих перекладів новинних статей.
- Дотренування нейронної мережі для перекладу тексту на нових даних. Задачою цього модулю є дотренування нейронної мережі для перекладу з української на англійську на нових даних. Результатом виконання цілей модулю буде натренована нейронна мережа для перекладу тексту, результат роботи якої буде потім передаватися в сумаризатор.
- Реалізація екстракції даних з новинних сайтів за тематикою та їх конвертація у формат, потрібний для моделі. Задачою цього модулю є створення функцій для скрейпінгу даних за тематикою з майже будь-яких новинних сайтів на різних мовах.
- Побудова сервісу, що поєднує всі вищезазначені функції. Ціллю даного модулю є створення кінцевого сервісу, що отримує на вхід посилання на джерело, мову та тематику і видає користувачеві сумаризацію новин з обраного сайту за обраною тематикою англійською мовою.

2.3 Конструювання програмного забезпечення

2.3.1 Введення в архітектуру трансформера

Цей підрозділ присвячений ознайомленню з архітектурою трансформера, яку буде імплементовано у даному програмному забезпеченні, та основними складовими, які зробили її лідуючою у сфері обробки природної мови.

Структура енкодера-декодера

До трансформерів рекурентні архітектури, такі як LSTM, були найсучаснішими в NLP. Ці архітектури містять петлю зворотного зв'язку в мережових з'єднаннях, що дозволяють інформації поширюватися від одного кроку до іншого, що робить їх ідеальними для моделювання послідовних даних, таких як текст. RNN отримує певний вхід (який може бути словом або символом), подає його через мережу та виводить вектор, який називається прихованим станом. У той же час модель повертає деяку інформацію собі через петлю зворотного зв'язку, яку потім може використовувати на наступному кроці. Це дозволяє RNN відстежувати інформацію з попередніх кроків і використовувати її для прогнозування вихідних даних.

Однією з областей, де RNN відігравали важливу роль, була розробка систем машинного перекладу, де метою є відображення послідовності слів однієї мови на іншу. Такого роду завдання зазвичай вирішуються за допомогою архітектури енкодера-декодера, яка добре підходить для ситуацій, коли вхід і вихід є послідовностями довільної довжини. Завдання енкодера полягає в тому, щоб закодувати інформацію з вхідної послідовності в числове представлення, яке часто називають останнім прихованим станом. Потім цей стан передається в декодер, який генерує вихідну послідовність.

Загалом, компонентами енкодера та декодера можуть бути будь-які архітектури нейронної мережі, які можуть моделювати послідовності.

Один з недоліків цієї архітектури полягає в тому, що кінцевий прихований стан енкодера створює так званий *information bottleneck*: він повинен представляти значення всієї вхідної послідовності, оскільки це все, до чого декодер має доступ під час генерування вихідних даних. Це становить особливу проблему для довгих послідовностей, де інформація на початку послідовності може бути втрачена в процесі стиснення всього до єдиного фіксованого представлення.

З цього є вихід, що дозволяє декодеру мати доступ до всіх прихованих станів енкодера. Загальний механізм цього називається увагою, і він є ключовим компонентом багатьох сучасних архітектур нейронних мереж.

Механізми уваги

Основна ідея уваги полягає в тому, що замість створення єдиного прихованого стану для вхідної послідовності, енкодер виводить прихований стан на кожному кроці, до якого може отримати доступ декодер. Однак використання всіх станів одночасно створить величезний вхід для декодера, тому потрібен певний механізм, щоб визначити пріоритетність станів. Саме тут на допомогу приходить механізм уваги: він дозволяє декодеру призначати різну вагу або «увагу» кожному зі станів енкодера на кожному етапі декодування. Цей процес проілюстрований на малюнку 2.1, де показано роль уваги для передбачення третього маркера у вихідній послідовності.

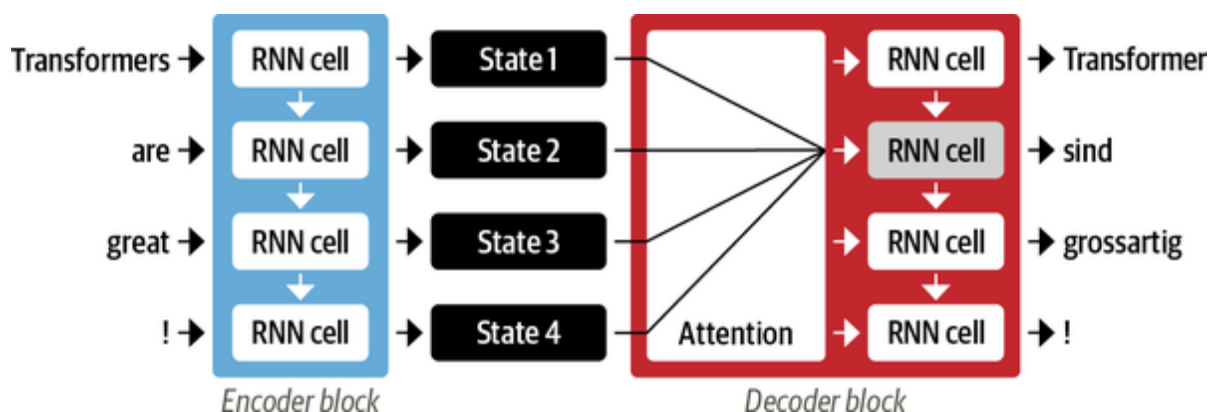


Рис. 2.2 Механізм уваги

Зосереджуючись на тому, які вхідні лексеми є найбільш релевантними на кожному етапі часу, моделі, що базуються на механізмі уваги, можуть

розпізнавати нетривіальні зв'язки між словами у перекладі та словами у початковому реченні.

Хоча механізм уваги дозволив створювати набагато кращі переклади, все ще був серйозний недолік у використанні рекурентних моделей для енкодера і декодера: обчислення за своєю суттю є послідовними і не можуть бути розпаралеленими. З трансформерами була представлена нова парадигма моделювання: повністю відмовитися від повторення, а натомість повністю покладатися на особливу форму уваги, яка називається самоувагою, ідея якої полягає в тому, щоб дозволити увазі оперувати всіма станами в одному шарі нейронної мережі. Це показано на малюнку 2.2, де і енкодер, і декодер мають власні механізми самоуваги, вихідні дані яких надходять до нейронних мереж із прямим зв'язком. Цю архітектуру можна навчити набагато швидше, ніж рекурентні моделі, і вона проклала шлях до багатьох останніх проривів у NLP.

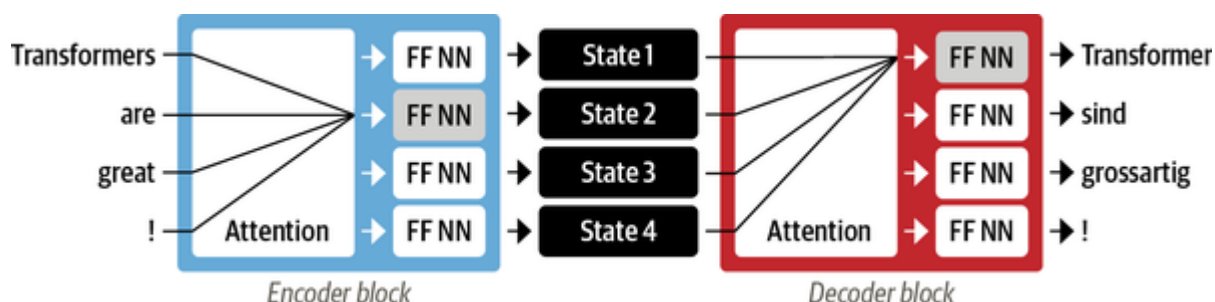


Рис 2.3 Механізм самоуваги

Трансформер заснований на архітектурі енкодера-декодера, яка широко використовується для таких завдань, як машинний переклад, коли послідовність слів перекладається з однієї мови на іншу. Ця архітектура складається з двох компонентів: енкодер, що перетворює вхідну послідовність маркерів у послідовність векторів вбудовування, яку часто називають прихованим станом або контекстом, та декодер, що використовує прихований стан енкодера, щоб ітеративно генерувати вихідну послідовність маркерів, по одному маркеру за раз.

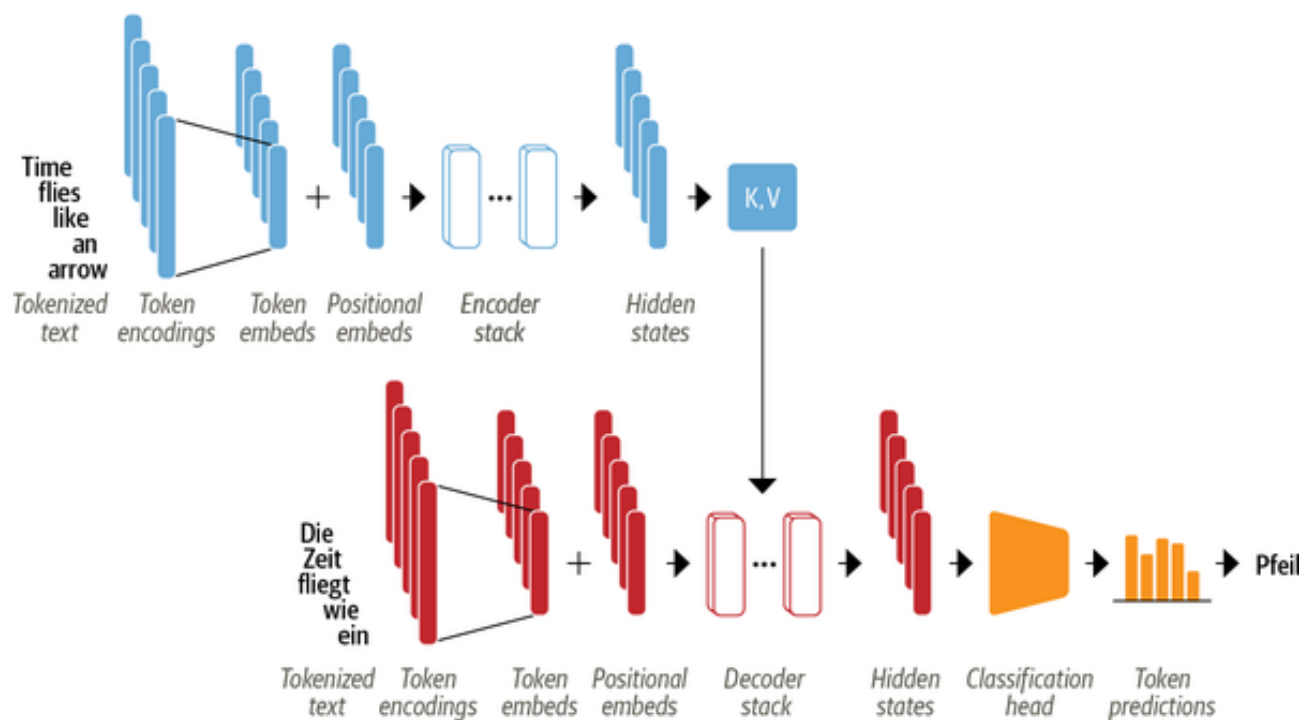


Рис 2.4 Архітектура трансформера

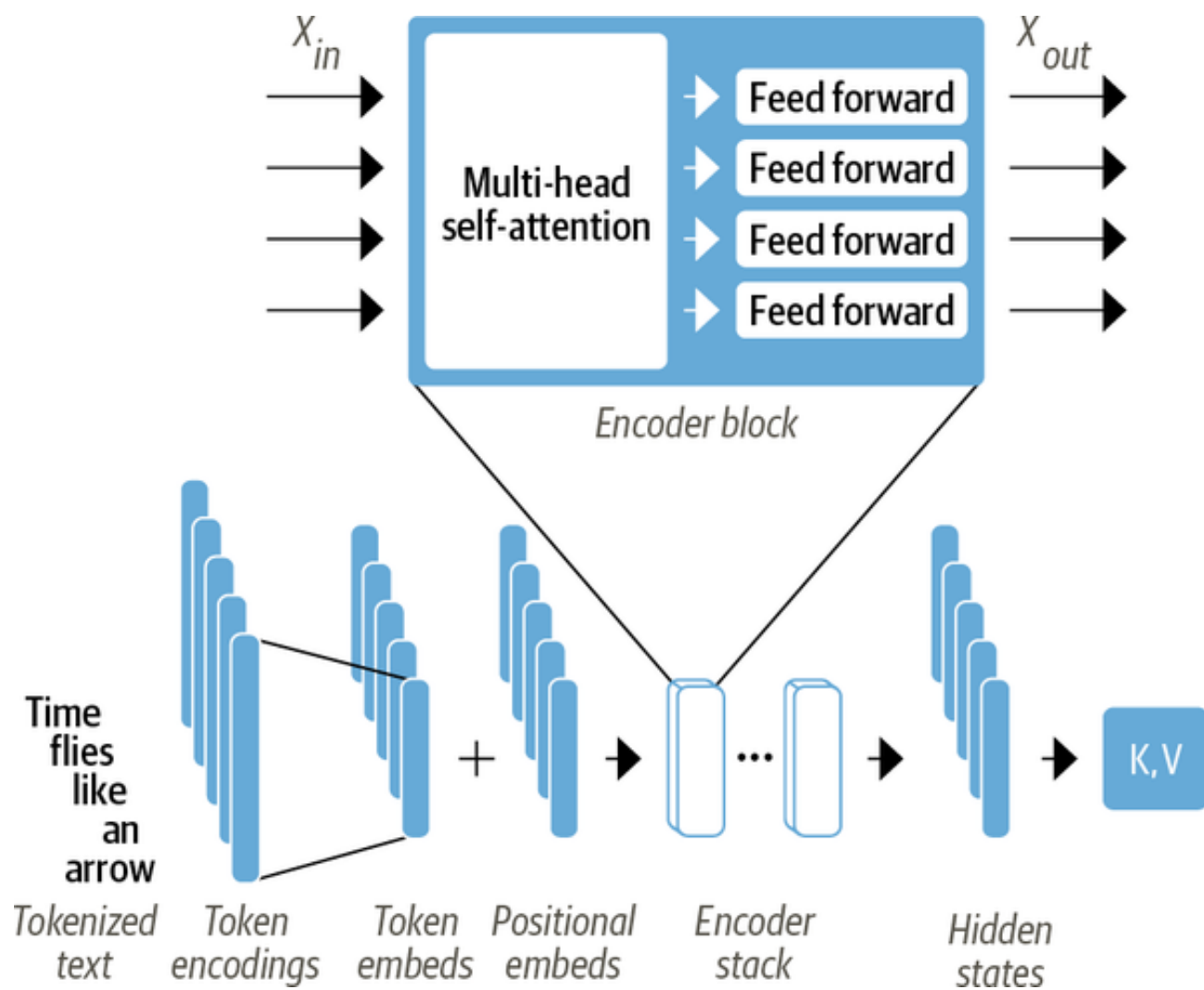


Рис 2.5 Архітектура блока енкодера

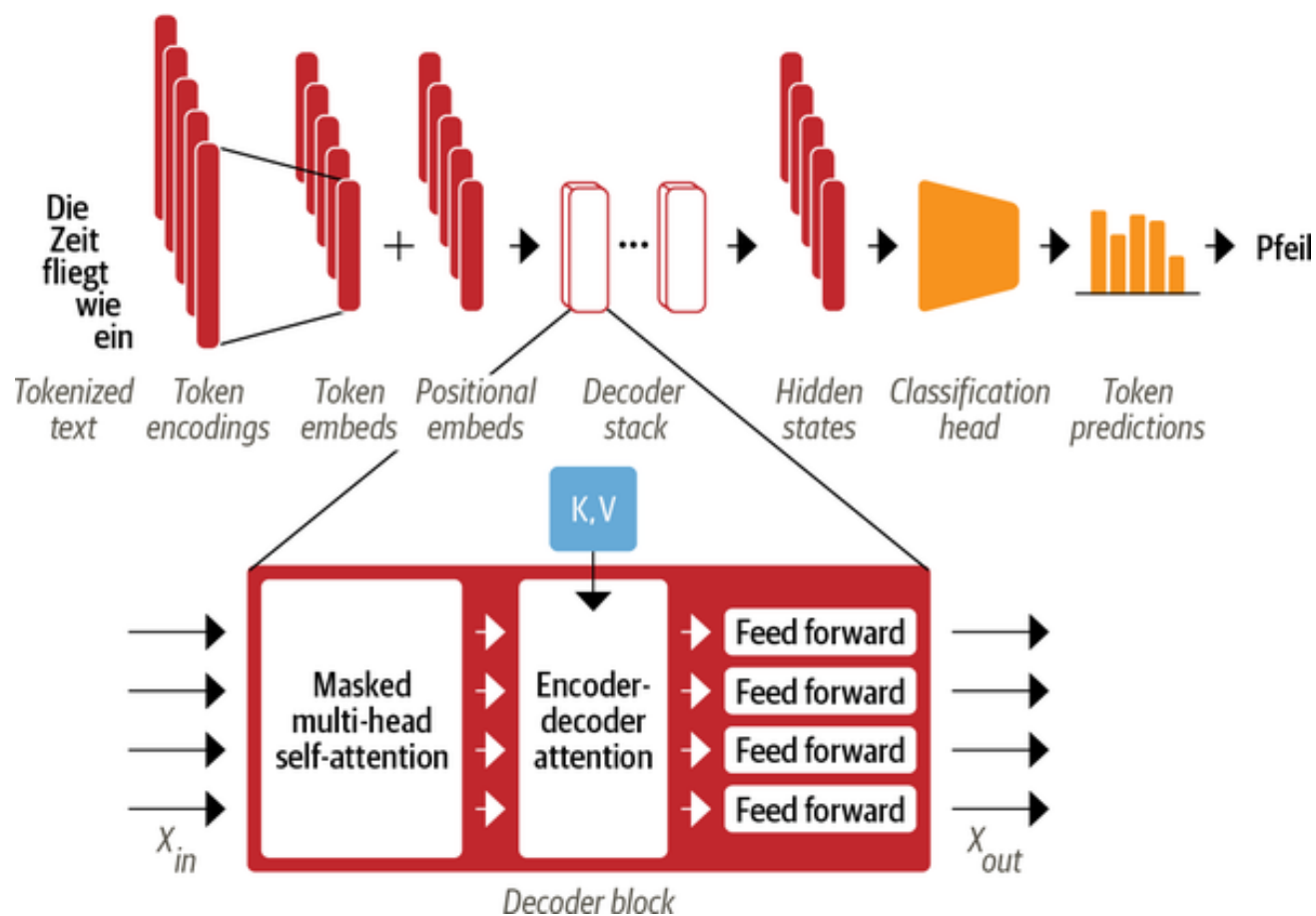


Рис 2.6 Архітектура блока декодера

2.3.2 Основні функції та класи.

Для створення модулю, що відповідає за побудову сумаризаційної моделі нам треба реалізувати архітектуру трансформера. Це включає в себе описані далі функції та класи.

Спочатку треба реалізувати функції що відповідають за попередню обробку, токенизацію та кодування вхідного тексту в ембедінги.

Потім йде реалізація багатоголового механізму самоуваги (multi-headed self-attention). Існує кілька способів реалізувати механізм самоуваги, але найпоширенішим з них є масштабована увага за точковим добутком (scaled dot product attention), з статті, що представляє архітектуру трансформера. Для реалізації цього механізму необхідні чотири основні кроки:

- Спроекувати кожен ембедінг в три вектори, які називаються query, key, value.
- Розрахувати показники уваги. Ми визначаємо, наскільки запит і ключові вектори пов'язані один з одним, використовуючи функцію подібності. Як випливає з назви, функція подібності для масштабованого добутку уваги — це крапковий добуток, який ефективно обчислюється за допомогою матричного множення ембедінгів. Подібні запити та ключі матимуть великий крапковий добуток, тоді як ті, які не мають спільного, майже не перетинатимуться. Результати цього кроку називаються оцінками уваги, а для послідовності з n вхідних ембедінгів існує відповідна матриця оцінок уваги.
- Обчислення ваги уваги. Оцінки уваги спочатку помножуються на коефіцієнт масштабування, щоб нормалізувати їх дисперсію, а потім нормалізуються за допомогою softmax, щоб забезпечити суму всіх значень стовпців до 1. Отримана матриця $n \times n$ містить усі вагові коефіцієнти уваги.

- Оновлення ембедінгів. Після обчислення ваги уваги ми множимо їх на вектор значень, щоб отримати оновлене представлення для ембедінгів.

Після цього ми можемо реалізувати енкодер та декодер і побудувати нейронну мережу.

Для модулю, ціллю якого є переклад тексту, нам треба знайти модель, що підходить для дотренування, знайти датасет, за допомогою якого воно буде відбуватись, натренувати модель та оцінити результати тренування за допомогою обраних метрик.

Реалізація модулю, що відповідає за обробку новинних сайтів, включає в себе функції, які вилучають з зазначених сайтів основні компоненти та проводять парсинг статей за обраною тематикою.

Останній модуль має на увазі об'єднання всього функціоналу у один веб-додадок, створений за допомогою фреймворку Flask та розгортання його на платформі AWS.

У базі даних зберегатимуться лише аутентифікаційні дані користувачів (це одна таблиця зі стовбцями username та password).

На наступних таблицях наведено опис основних функцій та класів для кожного з модулів сервісу

Таблиця 2.1 Основні класи та функції модулю, що відповідає за побудову сумаризаційної моделі

Назва	Опис
get_positional_encoding	Ця функція відповідає за отримання позиційного кодування для вхідних послідовностей. Позиційне кодування в основному впроваджує поняття порядку вхідних слів
make_padding_mask	Відповідає за ігнорування паддінгу, доданого до послідовностей,

	коротших за довжину максимальної послідовності
Продовження Таблиці 2.1	
make_lookahead_mask	Відповідає за ігнорування слів, які зустрічаються після поточного слова в цільовій послідовності
scaled_dotproduct_attention	Реалізація scaled dot product механізму уваги
feed_forward_network	Два слої нейронної мережі, що вставляються після механізму уваги
MultiheadAttention	Розбиває вхідні дані на кілька голів, обчислює ваги уваги, використовуючи scaled dot product attention, і об'єднує вихідні дані з усіх голів
EncoderBlock	Слой енкодера нейронної мережі
DecoderBlock	Слой декодера нейронної мережі
Encoder	Повноцінний енкодер, може містити декілька слоїв енкодера
Decoder	Повноцінний декодер, може містити декілька слоїв декодера
Transformer	Трансформер з енкодером та декодером

Таблиця 2.2 Основні класи та функції модулю, що відповідає за переклад

Назва	Опис
do_preprocessing	Попередня обробка та токенизація даних для тренування моделі
do_postprocessing	Конвертація результатів

get_metrics	Розрахування метрик для оцінки якості моделі
perform_fine_tuning	Дотренування моделі

Продовження Таблиці 2.2	
Назва	Опис
build_newspaper	Екстракція основних елементів новинного джерела за посиланням
extract_articles_by_topic	Відбір статей за тематикою
convert_articles_to_input_format	Трансформація статей до вигляду, що підходить для передачі моделі

2.4 Аналіз безпеки даних

Flask-Security дозволяє швидко додавати загальні механізми безпеки до програми. Вони включають аутентифікацію на основі сеансу, хешування паролей та базову аутентифікацію HTTP. Це буде реалізовано завдяки інтеграції різноманітних розширень і бібліотек Flask, таких як Flask-Login, Flask-Mail, Flask-Principal, Flask-WTF, itsdangerous, passlib. Загалом, додаток не зберігає чутливої особистої інформації про користувача.

Висновки до розділу

У даному розділі ми проаналізували вимоги і дійшли до висновку щодо архітектури програмного забезпечення та методів її імплементації. Для побудови сумаризаційної нейронної мережі було обрано фреймворк Tensorflow, для перекладу вирішено використовувати інструменти, що пропонує HuggingFace та каталог моделей-трансформерів, що пропонує HuggingFace Hub, для розгортання сервісу вирішено використовувати мікрофреймворк для вебдодатків Flask, та фреймворк для розгортання демо моделей машинного навчання Gradio. Для хостингу буде використовуватись платформа AWS. Також у модулі було описано кроки побудови сервісу.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		44

3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості ПЗ

У даному програмному забезпеченні наявні декілька модулів, що потребують окремого тестування, а отже аналіз якості програмного забезпечення буде складатися з декількох компонент, а саме: аналіз точності моделі для сумаризації на основі підібраних метрик, аналіз точності моделі для перекладу та тестування сервісу.

3.2 Опис процесів тестування

Для оцінки якості моделей для перекладу буде використано таку метрику як BLEU.

BLEU (BiLingual Evaluation Understudy) — це показник для автоматичної оцінки машинно перекладеного тексту. Оцінка BLEU – це число від нуля до одиниці, яке вимірює подібність машинно перекладеного тексту до набору високоякісних довідкових перекладів. Значення 0 означає, що результат машинного перекладу не збігається з еталонним перекладом (низька якість), тоді як значення 1 означає, що є повне перекриття з еталонним перекладом (висока якість). Оцінки BLEU добре корелюють з людським судженням про якість перекладу. Зауважте, що навіть люди-перекладачі не досягають ідеальної оцінки 1.

Ідея BLEU проста: замість того, щоб дивитися на те, скільки лексем у згенерованих текстах ідеально узгоджено з перекладом зробленим людиною, ми дивимося на слова або n-грами. BLEU — це метрика, заснована на точності, що означає, що коли ми порівнюємо два тексти, ми підраховуємо кількість слів у згенерованому тексті, які зустрічаються в тексті, на основі якого ми оцінюємо якість перекладу, і ділимо її на довжину згенерованого тексту.

Однак з цим варіантом точності є проблема. Припустимо, що згенерований текст просто повторює одне й те саме слово знову і знову, і це слово також є в довідковому матеріалі. Якщо його повторити стільки разів, скільки складає довжина довідкового тексту, ми отримаємо ідеальну точність. З цієї причини автори статті BLEU внесли невелику модифікацію: слово враховується лише стільки разів, скільки воно зустрічається в посиланні.

$$\text{BLEU} = \underbrace{\min\left(1, \exp\left(1 - \frac{\text{reference-length}}{\text{output-length}}\right)\right)}_{\text{brevity penalty}} \underbrace{\left(\prod_{i=1}^4 \text{precision}_i\right)^{1/4}}_{\text{n-gram overlap}}$$

$$\text{precision}_i = \frac{\sum_{\text{snt} \in \text{Cand-Corpus}} \sum_{i \in \text{snt}} \min(m_{\text{cand}}^i, m_{\text{ref}}^i)}{w_t^i = \sum_{\text{snt}' \in \text{Cand-Corpus}} \sum_{i' \in \text{snt}'} m_{\text{cand}}^{i'}}$$

Рис 3.1 Формула розрахунку метрики BLEU

У наведеній формулі m^i є кількістю разів, що зустрічається певна i -грама (cand — у машинному перекладі, ref — у перекладі, зробленому людиною, з яким порівнюється машинний переклад), w^i — загальна кількість i -грам у машинному перекладі. Штраф за стислість (brevity penalty) штрафує створені переклади, які є занадто короткими порівняно з найближчими референтними з експоненційним спадом. Штраф за стислість компенсує той факт, що оцінка BLEU не має терміну відкликання. Перекриття N-грам (n-gram overlap) підраховує, скільки уніграмів, біграмів, триграм і чотирьох грамів ($i=1, \dots, 4$) відповідають їх n-грамовому аналогу в еталонних перекладах. Цей термін виступає як точний показник. Уніграми враховують адекватність, а довші n-грами — плавність перекладу. Щоб уникнути надмірного підрахунку, кількість

n-грамів обрізається до максимальної кількості n-грам, що зустрічається в референтному перекладі.

Якість сумаризації буде оцінена в першу чергу з точки зору її змістовності та консистентності. Звичайні показники, такі як точність, не відображають якість згенерованого тексту.

Одною з метрики для оцінки якості узагальнення тексту буде ROUGE. Оцінка ROUGE була спеціально розроблена для оцінки таких речей як узагальнення, де високий рівень запам'ятовування важливіший, ніж просто точність. Підхід дуже схожий на оцінку BLEU, оскільки ми розглядаємо різні n-грами та порівнюємо їх входження в згенерований текст і довідковий текст. Різниця полягає в тому, що за допомогою ROUGE ми перевіряємо, скільки n-грам у довідковому тексті також зустрічається в згенерованому тексті. Для BLEU ми дивилися, скільки n-грамів у згенерованому тексті з'являється у довідковому. Показники BLEU та ROUGE можуть оцінювати згенеровані тексти, однак людське судження залишається найкращим критерієм.

Для тестування сервісу буде використано фреймворк pytest.

					КПІ.ІП-8228.045440.02.81	Арк.
						47
Змін.	Арк.	№ докум.	Підп.	Дата.		

Повинні бути проведені наступні тести:

Таблиця 3.1 Контрольний приклад 1

Назва тесту	Авторизація користувача
Мета тесту	Перевірка програмного забезпечення на можливість вдалої авторизації
Сценарій проведення тесту	Введення коректних даних аутентифікації
Очікуваний результат	Користувач входить у систему

Таблиця 3.2 Контрольний приклад 2

Назва тесту	Авторизація користувача при невірних даних
Мета тесту	Перевірка програмного забезпечення на коректність обробки невдалої авторизації
Сценарій проведення тесту	Введення некоректних даних аутентифікації
Очікуваний результат	Користувач отримує повідомлення про невірні аутентифікаційні дані

Таблиця 3.3 Контрольний приклад 3

Назва тесту	Введення коректного новинного джерела
Мета тесту	Перевірка програмного забезпечення на можливість обробляти існуючі новинні сайти
Сценарій проведення тесту	Введення коректних даних
Очікуваний результат	Відбувається парсинг обраного сайту

Таблиця 3.4 Контрольний приклад 4

Назва тесту	Введення некоректного новинного джерела
Мета тесту	Перевірка програмного забезпечення на коректність обробки помилки при введенні некоректної адреси або адреси, що не є новинним джерелом
Сценарій проведення тесту	Введення некоректних даних
Очікуваний результат	Користувач отримує повідомлення про помилку

Таблиця 3.5 Контрольний приклад 5

Назва тесту	Обирання мови новинного джерела
Мета тесту	Перевірка програмного забезпечення на можливість задання користувачем фільтрів для пошуку новинних статей
Сценарій проведення тесту	Обирання мови зі списку
Очікуваний результат	Фільтр застосовано

Таблиця 3.6 Контрольний приклад 6

Назва тесту	Обирання тематики новин
Мета тесту	Перевірка програмного забезпечення на можливість задання користувачем фільтрів для пошуку новинних статей
Сценарій проведення тесту	Вписання теми у відповідне поле
Очікуваний результат	Фільтр застосовано

Таблиця 3.7 Контрольний приклад 7

Назва тесту	Переклад та сумаризація тексту
Мета тесту	Перевірка програмного забезпечення на можливість перекладати та сумаризувати текст з обраного джерела інформації
Сценарій проведення тесту	Задання фільтрів, натискання на кнопку сумаризації
Очікуваний результат	Отримання сумаризації тексту англійською мовою

Висновки до розділу

У даному розділі було проаналізовано методи, що потрібно застосувати для перевірки якості програмного забезпечення та було з'ясовано шляхи їх імплементації. Для оцінки моделей будуть використовуватись метрики BLEU та ROUGE, додаток буде протестовано за допомогою вбудованого у Flask інструментарію. Також було розглянуто контрольні приклади, необхідні для перевірки якості програмного забезпечення.

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання програмного забезпечення

Для розгортання програмного забезпечення було обрано платформу AWS та EC2-інстанс. Після розгортання сервіс буде доступний користувачам за посиланням.

Amazon Elastic Compute Cloud (Amazon EC2) забезпечує масштабовані обчислювальні потужності в хмарі Amazon Web Services (AWS). Amazon EC2 дає змогу контролювати масштабованість та поратись зі змінами вимог до сервісу та перепадами трафіку.

Amazon EC2 надає такі ресурси:

- Віртуальні обчислювальні середовища (instances)
- Попередньо налаштовані шаблони для цих середовищ (Amazon Machine Images)
- Різні конфігурації процесорів, обсягу пам'яті та налаштування мережі для різних інстансів
- Безпечна аутентифікація за допомогою пар ключів (AWS зберігає відкритий ключ, а користувач зберігає закритий ключ у захищеному місці)
- Сховища для тимчасових даних, які видаляються під час зупинки, сплячого режиму або завершення роботи інстансу
- Зберігання даних за допомогою Amazon Elastic Block Store (Amazon EBS)
- Кілька фізичних місць розташування ресурсів, відомі як регіони та зони доступності

- Брандмауер, який дозволяє вказувати протоколи, порти та діапазони вихідних IP-адрес, які можуть досягати ваших інстансів, за допомогою груп безпеки
- Статичні IPv4-адреси для динамічних хмарних обчислень
- Віртуальні мережі, логічно ізольовані від решти хмари AWS, і які за бажанням можна підключити до власної мережі (VPC).

4.2 Робота з програмним забезпеченням

Програмне забезпечення дозволяє користувачеві отримувати сумаризацію новин за темою з обраного джерела. Щоб почати взаємодіяти з сервісом користувач потрібен пройти аутентифікацію. Після цього він може налаштувати фільтри пошуку статей у відповідних полях та запросити узагальнення новин. Більш детальна інструкція знаходиться у додатку «Інструкція з користування»

Висновки до розділу

У даному розділі було визначено, що розгортання програмного забезпечення буде здійснено за допомогою платформи Amazon Web Services і EC2-інстанса та описано роботу зі створюваним сервісом.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		52

ВИСНОВКИ

Сьогодні питання глобального доступу до інформації є як ніколи актуальним, тому створення сервісу, що дозволить отримувати короткий зміст новин з різних джерел мовою користувача є досить корисним. Існує досить багато відомих рішень, що дозволяють виконувати переклад або сумаризацію тексту, але немає відомих інструментів, заточених під взаємодію з новинними джерелами та поєднуючих зазначені функції. На даний момент домінуючою архітектурою нейронних мереж для перекладу та сумаризації тексту є трансформери, представлені у 2017 році у статті “Attention is all you need”.

Для побудови сумаризаційної нейронної мережі було обрано фреймворк Tensorflow, для перекладу вирішено використовувати інструменти, що пропонує HuggingFace та каталог моделей-трансформерів, що пропонує HuggingFace Hub, для розгортання сервісу вирішено використовувати мікрофреймворк для вебдодатків Flask, та фреймворк для розгортання демо моделей машинного навчання Gradio. Для хостингу було використано платформу AWS.

У результаті роботи над проектом було створено програмне забезпечення з користувацьким інтерфейсом для перекладу новин з різних інформаційних джерел на англійську та їх сумаризації за допомогою моделей глибокого навчання.

Для сумаризації тексту було побудовано нейронну мережу з архітектурою трансформера, що на даний момент є найефективнішою для задач обробки природної мови з відомих архітектур. Для перекладу було використано існуючу модель-трансформер та здійснено її fine-tuning на нових даних. Результати роботи моделі було покращено.

За допомогою фреймворку Flask та платформи AWS було побудовано та розгорнуто веб-додаток, що дозволяє користувачам виконувати переклад та сумаризацію новин з обраних джерел за бажаною тематикою. Було оцінено якість моделей та протестовано побудований додаток.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		53

Створений проект має можливості для подальшого розвитку. Однією з найочевидніших перспектив розширення сервісу є додання підтримки більшої кількості мов. Кінцева мета — отримувати короткий зміст новин з будь-якого джерела майже будь-якою мовою. Також можливе додання таких функцій для проведення інформаційної аналітики як аналіз настрою або порівняння змісту.

Отже, поставлені задачі виконано.

					КПІ.ІП-8228.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		54

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Leandro von Werra, Lewis Tunstall, Thomas Wolf Natural Language Processing with Transformers. Sebastopol, 2022. 408 p.
2. Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems. Sebastopol, 2020. 456 p.
3. Hugging Face Integrations: [Веб-сайт]. URL:
https://gradio.app/using_hugging_face_integrations/
4. Newspaper3k: [Веб-сайт].
URL: <https://newspaper.readthedocs.io/en/latest/>
5. Understanding LSTMs: [Веб-сайт] URL:
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
6. An Introduction To Recurrent Neural Networks And The Math That Powers Them: [Веб-сайт]. URL:
<https://machinelearningmastery.com/an-introduction-to-recurrent-neural-networks-and-the-math-that-powers-them/>
7. Reasons to choose Flask: [Веб-сайт]. URL:
<https://able.bio/hardikshah/6-reasons-why-flask-is-better-framework-for-web-application-development--cd398f73>
8. Transformers Explained: [Веб-сайт]. URL:
<https://towardsdatascience.com/transformers-explained-65454c0f3fa7>



Имя пользователя:
Лісовиченко Олег Іванович

Дата проверки:
17.06.2022 19:16:34 EEST

Дата отчета:
17.06.2022 19:17:10 EEST

ID проверки:
1011606500

Тип проверки:
Doc vs Internet + Library

ID пользователя:
76913

Название файла: **ІП-82_Рибалко_ПЗ**

Количество страниц: 49 Количество слов: 6698 Количество символов: 53334 Размер файла: 1.39 MB ID файла: 1011475011

9.27% Совпадения

Наибольшее совпадение: 1.87% с источником из Библиотеки (ID файла: 1009544895)

1.81% Источники из Интернета 6 Страница 51

9.14% Источники из Библиотеки 168 Страница 51

0% Цитат

Исключение цитат выключено

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПЕРЕКЛАДУ ТА СУМАРИЗАЦІЇ
НОВИН**

Технічне завдання

КПІ.ПІ-8228.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Сергій Лебідь

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Анастасія РИБАЛКО

Київ – 2022

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2 ПІДСТАВА ДЛЯ РОЗРОБКИ	4
3 ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1 Вимоги до функціональних характеристик.....	6
4.2 Вимоги до надійності.....	6
4.3 Умови експлуатації	6
4.4 Вимоги до складу і параметрів технічних засобів.....	7
4.5 Вимоги до інформаційної та програмної сумісності.....	7
4.6 Вимоги до маркування та пакування	7
4.7 Вимоги до транспортування та зберігання.....	7
4.8 Спеціальні вимоги.....	7
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	9
6 СТАДІЇ І ЕТАПИ РОЗРОБКИ	11
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	12

5 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Програмне забезпечення для перекладу та сумаризації новин.

Галузь застосування: дане програмне призначено для полегшення доступу до інформації та зменшення часу на її обробку. Дозволяє користувачам здійснювати переклад та сумаризацію новин з обраних джерел.

Наведене технічне завдання поширюється на розробку програмного забезпечення «Програмне забезпечення для перекладу та сумаризації новин» [КПІ.ІП-8123.045440.01.91], котра використовується для здійснення перекладу та сумаризації новин та призначена для користування пересічними людьми у повсякденному житті.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

6 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки «Програмне забезпечення для перекладу та сумаризації новин» є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

7 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для полегшення доступу до інформації з новинних джерел на різних мовах.

Метою розробки є створення інструменту, що дозволить здійснювати переклад та сумаризацію новин за допомогою нейронних мереж.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

8 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

8.1 Вимоги до функціональних характеристик

8.1.1 Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

8.1.1.1 Для користувача:

- налаштування фільтрів для пошуку новинних статей;
- переклад і сумаризація тексту за обраними фільтрами

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

8.1.2 Розробку виконати на мові програмування Python за допомогою фреймворків Tensorflow, HuggingFace, Gradio, Flask.

8.2 Вимоги до надійності

8.2.1 Передбачити контроль введення інформації.

8.2.2 Передбачити захист від некоректних дій користувача.

8.2.3 Забезпечити цілісність інформації в базі даних.

8.3 Умови експлуатації

8.3.1 Умови експлуатації згідно СанПін 2.2.2.542 – 96.

8.4 Вимоги до складу і параметрів технічних засобів

Вимог до складу і параметрів технічних засобів немає.

8.4.1 Мінімальна конфігурація технічних засобів:

8.4.1.1 Тип процесору Pentium.

8.4.1.2 Об'єм ОЗП 32 Мб.

8.5 Вимоги до інформаційної та програмної сумісності

8.5.1 Програмне забезпечення повинно працювати під управлінням операційних си-стем сімейства WIN32 (Windows'XP, Windows NT і т.д.) або Unix.

8.5.2 Вхідні дані повинні бути представлені в наступному форматі: налаштування фільтрів на сайті.

8.5.3 Результати повинні бути представлені в наступному форматі:
текст

8.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не пред'являються.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

8.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не пред'являються.

8.8 Спеціальні вимоги

Згенерувати установчу версію програмного забезпечення.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

9 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

9.1 Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

9.2 Програмне забезпечення повинно мати довідникову систему.

9.3 У склад супроводжувальної документації повинні входити наступні документи:

9.3.1 Пояснювальна записка не менше ніж на 50 аркушах формату А4 (без додатків 5.3.2 - 5.3.6).

9.3.2 Технічне завдання.

9.3.3 Керівництво користувача

9.3.4 Програма та методика тестування

9.4 Графічна частина повинна бути виконана не менше ніж на 3 аркушах формату А3, котрі включаються у якості додатків до пояснювальної записки:

9.4.1 Схема структура інформаційної системи.

9.4.2 Схема структурна програмного забезпечення.

9.4.3 Схема функціональна програмного забезпечення.

9.4.4 Схема структура потоків даних програмного забезпечення або його частини.

9.4.5 Схема структурна компонентів структур даних

9.4.6 Схема структурна варіантів використання

9.4.7 Схема структурна концептуальної моделі предметного середовища

9.4.8 Схеми взаємодії об'єктів, об'єктна декомпозиція.

9.4.9 Схема структурна компонент.

9.4.10 Схема структурна класів програмного забезпечення

9.4.11 Схема структурна станів інтерфейсу

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

9.4.12 Креслення вигляду екранних форм.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

10 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проекту	10.03	
2.	Розробка технічного завдання	15.04	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	07.05	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	30.03	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Програмна реалізація програмного забезпечення	22.05	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	30.05	Тести, результати тестування
7.	Розробка матеріалів текстової частини проекту	07.06	Пояснювальна записка
8.	Розробка матеріалів графічної частини проекту	02.06	Графічний матеріал проекту
9.	Оформлення технічної документації проекту	10.06	Технічна документація

11 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

					КПІ.ІП-8228.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		12

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

Програмне забезпечення для перекладу та сумаризації новин

Керівництво користувача

КПІ.ПІ-8228.045440.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Катерина ХОЛОЛОВИЧ

Київ – 2022

ЗМІСТ

1 Загальні відомості	3
2 Підготовка до роботи.....	4
2.1 Системні вимоги для коректної роботи	4
2.2 Завантаження застосунку.....	4
2.3 Перевірка коректної роботи.....	4
3 Робота із застосунком.....	5

					КПІ.ІП-8228.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 ЗАГАЛЬНІ ВІДОМОСТІ

«Програмне забезпечення для перекладу та сумаризації новин» - це веб-застосунок, призначений для здійснення перекладу та узагальнення змісту новин за обраною тематикою з різних новинних джерел.

					КПІ.ІП-8228.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДГОТОВКА ДО РОБОТИ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- наявність доступу до Інтернету;

2.2 Завантаження застосунку

Застосунок не треба завантажувати.

2.3 Перевірка коректної роботи

Сторінка надає можливість користуватися фільтрами для новинних статей та робити запити на переклад та сумаризацію.

					КПІ.ІП-8228.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 РОБОТА ІЗ ЗАСТОСУНКОМ

При запуску програмного застосунку та авторизації користувач побачить сторінку з фільтрами для налаштування пошуку новин.

Спочатку користувачеві потрібно ввести адресу сайту новинного джерела, з якого він хоче взяти інформацію. Потім він повинен обрати з drop-down списку мову цього джерела (підтримуються лише мови, що є у drop-down списку). Після цього користувачеві треба зазначити у відповідному полі тематику, за якою він хоче шукати новини. Далі він натискає на кнопку сумаризації і отримує сумаризацію новин з обраного джерела за обраною тематикою на англійській мові.

					КПІ.ІП-8228.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ТЕМА ДИПЛОМНОГО ПРОЄКТУ

Опис програми

КПІ.ПІ-8228.045440.03.13

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Сергій ЛЕБІДЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Анастасія РИБАЛКО

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Програмне забезпечення для перекладу та сумаризації новин

(Найменування програми (документа))

CD-R

(Вид носія даних)

12 арк, 10100 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КПІ.ІП-8228.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

```
[ ] import pandas as pd
import numpy as np
import tensorflow as tf
import time
import re
import pickle
```

```
[ ] import tensorflow as tf
import tensorflow_datasets as tfds
```

```
[ ] !pip install --upgrade tensorflow tensorflow_datasets
```

```
▶ cnn_news = tfds.load('cnn_dailymail')
```

Downloading and preparing dataset 558.32 MiB (download: 558.32 MiB, generated: 1.28 GiB, total: 1.82 GiB)

DL Completed...: 100% 5/5 [01:59<00:00, 6.44s/ url]

DL Size...: 557/? [01:59<00:00, 15.72 MiB/s]

Extraction completed...: 100% 2/2 [01:59<00:00, 57.92s/ file]

```
[ ] df['article'] = df['article'].apply(lambda x: x.decode('utf-8'))
```

```
[ ] df['highlights'] = df['highlights'].apply(lambda x: x.decode('utf-8'))
```

```
[ ] df['article'][0]
```

'By. Associated Press. PUBLISHED:. 14:11 EST, 25 October 2013. |. UPDATED:. 15:36 EST, 25 October 2013. The bishop of the Fargo Catholic Diocese in North Dakota has exposed hundreds of church members in Fargo, Grand Forks and Jamestown to the hepatitis A virus in late September and early October. The state Health Department has exposed people for anyone who attended five churches and took communion. Bishop John Folda (pictured) of the Fargo Catholic Diocese in North Dakota has exposed people to the possible exposure. The diocese announced on Monday that Bishop John Folda is taking time off after being diagnosed with hepatitis A. The diocese said that the exposure was through contaminated food while attending a conference for newly ordained bishops in...

```
[ ] document = df['article']
summary = df['highlights']
```

```
▶ for decoder sequence
```

```
summary = summary.apply(lambda x: '<go> ' + x + ' <stop>')
summary.head()
```

```
0 <go> Bishop John Folda, of North Dakota, is ta...
1 <go> Criminal complaint: Cop used his role to ...
2 <go> Craig Eccleston-Todd, 27, had drunk at le...
3 <go> Nina dos Santos says Europe must be ready...
4 <go> Fleetwood top of League One after 2-0 win...
Name: highlights, dtype: object
```

					КПІ.ІП-8228.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

```
[ ] # since < and > from default tokens cannot be removed
filters = '!"#$%&()*+,-./:;<=?@[\]^_`{|}~\t\n'
oov_token = '<unk>'

[ ] document_tokenizer = tf.keras.preprocessing.text.Tokenizer(oov_token=oov_token)
summary_tokenizer = tf.keras.preprocessing.text.Tokenizer(filters=filters, oov_token=oov_token)

[ ] document_tokenizer.fit_on_texts(document)
summary_tokenizer.fit_on_texts(summary)

[ ] inputs = document_tokenizer.texts_to_sequences(document)
targets = summary_tokenizer.texts_to_sequences(summary)

[ ] summary_tokenizer.texts_to_sequences(["This is a test"])

[[56, 13, 8, 551]]

[ ] summary_tokenizer.sequences_to_texts([[184, 22, 12, 71]])

['manchester her on s']

[ ] encoder_vocab_size = len(document_tokenizer.word_index) + 1
decoder_vocab_size = len(summary_tokenizer.word_index) + 1

# vocab_size
encoder_vocab_size, decoder_vocab_size

(785451, 230200)
```

```
[ ] document_lengths = pd.Series([len(x) for x in document])
summary_lengths = pd.Series([len(x) for x in summary])

[ ] encoder_maxlen = 400
decoder_maxlen = 75

[ ] inputs = tf.keras.preprocessing.sequence.pad_sequences(inputs, maxlen=encoder_maxlen, padding='post', truncating='post')
targets = tf.keras.preprocessing.sequence.pad_sequences(targets, maxlen=decoder_maxlen, padding='post', truncating='post')

[ ] inputs = tf.cast(inputs, dtype=tf.int32)
targets = tf.cast(targets, dtype=tf.int32)

[ ] BUFFER_SIZE = 20000
BATCH_SIZE = 64

[ ] dataset = tf.data.Dataset.from_tensor_slices((inputs, targets)).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

[ ] def get_angles(position, i, d_model):
    angle_rates = 1 / np.power(10000, (2 * (i // 2)) / np.float32(d_model))
    return position * angle_rates

[ ] def positional_encoding(position, d_model):
    angle_rads = get_angles(
        np.arange(position)[:], np.newaxis,
        np.arange(d_model)[np.newaxis, :],
        d_model
```

```
[ ] def positional_encoding(position, d_model):
    angle_rads = get_angles(
        np.arange(position)[: , np.newaxis],
        np.arange(d_model)[np.newaxis, :],
        d_model
    )

    # apply sin to even indices in the array; 2i
    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])

    # apply cos to odd indices in the array; 2i+1
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])

    pos_encoding = angle_rads[np.newaxis, ...]

    return tf.cast(pos_encoding, dtype=tf.float32)

[ ] def create_padding_mask(seq):
    seq = tf.cast(tf.math.equal(seq, 0), tf.float32)
    return seq[:, tf.newaxis, tf.newaxis, :]

[ ] def create_look_ahead_mask(size):
    mask = 1 - tf.linalg.band_part(tf.ones((size, size)), -1, 0)
    return mask

[ ] def scaled_dot_product_attention(q, k, v, mask):
    matmul_qk = tf.matmul(q, k, transpose_b=True)

    dk = tf.cast(tf.shape(k)[-1], tf.float32)

    scaled_attention_logits = matmul_qk / tf.math.sqrt(dk)

    if mask is not None:
        scaled_attention_logits += (mask * -1e9)

    attention_weights = tf.nn.softmax(scaled_attention_logits, axis=-1)

    output = tf.matmul(attention_weights, v)
    return output, attention_weights
```

```
[ ] def scaled_dot_product_attention(q, k, v, mask):
    matmul_qk = tf.matmul(q, k, transpose_b=True)

    dk = tf.cast(tf.shape(k)[-1], tf.float32)
    scaled_attention_logits = matmul_qk / tf.math.sqrt(dk)

    if mask is not None:
        scaled_attention_logits += (mask * -1e9)

    attention_weights = tf.nn.softmax(scaled_attention_logits, axis=-1)

    output = tf.matmul(attention_weights, v)
    return output, attention_weights

[ ] class MultiHeadAttention(tf.keras.layers.Layer):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        self.num_heads = num_heads
        self.d_model = d_model

        assert d_model % self.num_heads == 0

        self.depth = d_model // self.num_heads

        self.wq = tf.keras.layers.Dense(d_model)
        self.wk = tf.keras.layers.Dense(d_model)
        self.wv = tf.keras.layers.Dense(d_model)

        self.dense = tf.keras.layers.Dense(d_model)

    def split_heads(self, x, batch_size):
```

Змін.	Арк.	№ докум.	Підп.	Дата.

КПІ.ІП-8228.045440.03.13

Арк.

5

```

self.depth = d_model // self.num_heads

self.wq = tf.keras.layers.Dense(d_model)
self.wk = tf.keras.layers.Dense(d_model)
self.wv = tf.keras.layers.Dense(d_model)

self.dense = tf.keras.layers.Dense(d_model)

def split_heads(self, x, batch_size):
    x = tf.reshape(x, (batch_size, -1, self.num_heads, self.depth))
    return tf.transpose(x, perm=[0, 2, 1, 3])

def call(self, v, k, q, mask):
    batch_size = tf.shape(q)[0]

    q = self.wq(q)
    k = self.wk(k)
    v = self.wv(v)

    q = self.split_heads(q, batch_size)
    k = self.split_heads(k, batch_size)
    v = self.split_heads(v, batch_size)

    scaled_attention, attention_weights = scaled_dot_product_attention(
        q, k, v, mask)

    scaled_attention = tf.transpose(scaled_attention, perm=[0, 2, 1, 3])

    concat_attention = tf.reshape(scaled_attention, (batch_size, -1, self.d_model))
    output = self.dense(concat_attention)

    return output, attention_weights

```

```

[ ] def point_wise_feed_forward_network(d_model, dff):
    return tf.keras.Sequential([
        tf.keras.layers.Dense(dff, activation='relu'),
        tf.keras.layers.Dense(d_model)
    ])

```

```

class EncoderLayer(tf.keras.layers.Layer):
    def __init__(self, d_model, num_heads, dff, rate=0.1):
        super(EncoderLayer, self).__init__()

        self.mha = MultiHeadAttention(d_model, num_heads)
        self.ffn = point_wise_feed_forward_network(d_model, dff)

        self.layernorm1 = tf.keras.layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = tf.keras.layers.LayerNormalization(epsilon=1e-6)

        self.dropout1 = tf.keras.layers.Dropout(rate)
        self.dropout2 = tf.keras.layers.Dropout(rate)

    def call(self, x, training, mask):
        attn_output, _ = self.mha(x, x, x, mask)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(x + attn_output)

        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        out2 = self.layernorm2(out1 + ffn_output)

        return out2

```

Змін.	Арк.	№ докум.	Підп.	Дата.

КПІ.ІП-8228.045440.03.13

Арк.

6

```

class DecoderLayer(tf.keras.layers.Layer):
    def __init__(self, d_model, num_heads, dff, rate=0.1):
        super(DecoderLayer, self).__init__()

        self.mha1 = MultiHeadAttention(d_model, num_heads)
        self.mha2 = MultiHeadAttention(d_model, num_heads)

        self.ffn = point_wise_feed_forward_network(d_model, dff)

        self.layernorm1 = tf.keras.layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = tf.keras.layers.LayerNormalization(epsilon=1e-6)
        self.layernorm3 = tf.keras.layers.LayerNormalization(epsilon=1e-6)

        self.dropout1 = tf.keras.layers.Dropout(rate)
        self.dropout2 = tf.keras.layers.Dropout(rate)
        self.dropout3 = tf.keras.layers.Dropout(rate)

    def call(self, x, enc_output, training, look_ahead_mask, padding_mask):
        attn1, attn_weights_block1 = self.mha1(x, x, x, look_ahead_mask)
        attn1 = self.dropout1(attn1, training=training)
        out1 = self.layernorm1(attn1 + x)

        attn2, attn_weights_block2 = self.mha2(enc_output, enc_output, out1, padding_mask)
        attn2 = self.dropout2(attn2, training=training)
        out2 = self.layernorm2(attn2 + out1)

        ffn_output = self.ffn(out2)
        ffn_output = self.dropout3(ffn_output, training=training)
        out3 = self.layernorm3(ffn_output + out2)

        return out3, attn_weights_block1, attn_weights_block2

```

```

class Encoder(tf.keras.layers.Layer):
    def __init__(self, num_layers, d_model, num_heads, dff, input_vocab_size, maximum_position_encoding, rate=0.1):
        super(Encoder, self).__init__()

        self.d_model = d_model
        self.num_layers = num_layers

        self.embedding = tf.keras.layers.Embedding(input_vocab_size, d_model)
        self.pos_encoding = positional_encoding(maximum_position_encoding, self.d_model)

        self.enc_layers = [EncoderLayer(d_model, num_heads, dff, rate) for _ in range(num_layers)]
        self.dropout = tf.keras.layers.Dropout(rate)

    def call(self, x, training, mask):
        seq_len = tf.shape(x)[1]

        x = self.embedding(x)
        x *= tf.math.sqrt(tf.cast(self.d_model, tf.float32))
        x += self.pos_encoding[:, :seq_len, :]

        x = self.dropout(x, training=training)

        for i in range(self.num_layers):
            x = self.enc_layers[i](x, training, mask)

        return x

```



```
class Decoder(tf.keras.layers.Layer):
    def __init__(self, num_layers, d_model, num_heads, dff, target_vocab_size, maximum_position_encoding, rate=0.1):
        super(Decoder, self).__init__()

        self.d_model = d_model
        self.num_layers = num_layers

        self.embedding = tf.keras.layers.Embedding(target_vocab_size, d_model)
        self.pos_encoding = positional_encoding(maximum_position_encoding, d_model)

        self.dec_layers = [DecoderLayer(d_model, num_heads, dff, rate) for _ in range(num_layers)]
        self.dropout = tf.keras.layers.Dropout(rate)

    def call(self, x, enc_output, training, look_ahead_mask, padding_mask):
        seq_len = tf.shape(x)[1]
        attention_weights = {}

        x = self.embedding(x)
        x *= tf.math.sqrt(tf.cast(self.d_model, tf.float32))
        x += self.pos_encoding[:, :seq_len, :]

        x = self.dropout(x, training=training)

        for i in range(self.num_layers):
            x, block1, block2 = self.dec_layers[i](x, enc_output, training, look_ahead_mask, padding_mask)

            attention_weights['decoder_layer{}_block1'.format(i+1)] = block1
            attention_weights['decoder_layer{}_block2'.format(i+1)] = block2

        return x, attention_weights
```

```
[ ] class Transformer(tf.keras.Model):
    def __init__(self, num_layers, d_model, num_heads, dff, input_vocab_size, target_vocab_size, pe_input, pe_target, rate=0.1):
        super(Transformer, self).__init__()

        self.encoder = Encoder(num_layers, d_model, num_heads, dff, input_vocab_size, pe_input, rate)
        self.decoder = Decoder(num_layers, d_model, num_heads, dff, target_vocab_size, pe_target, rate)
        self.final_layer = tf.keras.layers.Dense(target_vocab_size)

    def call(self, inp, tar, training, enc_padding_mask, look_ahead_mask, dec_padding_mask):
        enc_output = self.encoder(inp, training, enc_padding_mask)

        dec_output, attention_weights = self.decoder(tar, enc_output, training, look_ahead_mask, dec_padding_mask)

        final_output = self.final_layer(dec_output)

        return final_output, attention_weights

[ ] num_layers = 4
    d_model = 256
    dff = 1024
    num_heads = 8
    EPOCHS = 20
```

```

class CustomSchedule(tf.keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self, d_model, warmup_steps=4000):
        super(CustomSchedule, self).__init__()

        self.d_model = d_model
        self.d_model = tf.cast(self.d_model, tf.float32)

        self.warmup_steps = warmup_steps

    def __call__(self, step):
        arg1 = tf.math.rsqrt(step)
        arg2 = step * (self.warmup_steps ** -1.5)

        return tf.math.rsqrt(self.d_model) * tf.math.minimum(arg1, arg2)

learning_rate = CustomSchedule(d_model)

optimizer = tf.keras.optimizers.Adam(learning_rate, beta_1=0.9, beta_2=0.98, epsilon=1e-9)

loss_object = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True, reduction='none')

def loss_function(real, pred):
    mask = tf.math.logical_not(tf.math.equal(real, 0))
    loss_ = loss_object(real, pred)

    mask = tf.cast(mask, dtype=loss_.dtype)
    loss_ *= mask

    return tf.reduce_sum(loss_)/tf.reduce_sum(mask)

```

```

[ ] transformer = Transformer(
    num_layers,
    d_model,
    num_heads,
    dff,
    encoder_vocab_size,
    decoder_vocab_size,
    pe_input=encoder_vocab_size,
    pe_target=decoder_vocab_size,
)

def create_masks(inp, tar):
    enc_padding_mask = create_padding_mask(inp)
    dec_padding_mask = create_padding_mask(inp)

    look_ahead_mask = create_look_ahead_mask(tf.shape(tar)[1])
    dec_target_padding_mask = create_padding_mask(tar)
    combined_mask = tf.maximum(dec_target_padding_mask, look_ahead_mask)

    return enc_padding_mask, combined_mask, dec_padding_mask

[ ] checkpoint_path = "checkpoints"

ckpt = tf.train.Checkpoint(transformer=transformer, optimizer=optimizer)

ckpt_manager = tf.train.CheckpointManager(ckpt, checkpoint_path, max_to_keep=5)

if ckpt_manager.latest_checkpoint:
    ckpt.restore(ckpt_manager.latest_checkpoint)

```

```
[ ] @tf.function
def train_step(inp, tar):
    tar_inp = tar[:, :-1]
    tar_real = tar[:, 1:]

    enc_padding_mask, combined_mask, dec_padding_mask = create_masks(inp, tar_inp)

    with tf.GradientTape() as tape:
        predictions, _ = transformer(
            inp, tar_inp,
            True,
            enc_padding_mask,
            combined_mask,
            dec_padding_mask
        )
        loss = loss_function(tar_real, predictions)

    gradients = tape.gradient(loss, transformer.trainable_variables)
    optimizer.apply_gradients(zip(gradients, transformer.trainable_variables))

    train_loss(loss)
```

```
▶ for epoch in range(EPOCHS):
    start = time.time()

    train_loss.reset_states()

    for (batch, (inp, tar)) in enumerate(dataset):
        train_step(inp, tar)

    if batch % 500 == 0:
```

```
[ ] for epoch in range(EPOCHS):
    start = time.time()

    train_loss.reset_states()

    for (batch, (inp, tar)) in enumerate(dataset):
        train_step(inp, tar)

        if batch % 500 == 0:
            print('Epoch {} Batch {} Loss {:.4f}'.format(epoch + 1, batch, train_loss.result()))

    if (epoch + 1) % 5 == 0:
        ckpt_save_path = ckpt_manager.save()
        print('Saving checkpoint for epoch {} at {}'.format(epoch+1, ckpt_save_path))

    print('Epoch {} Loss {:.4f}'.format(epoch + 1, train_loss.result()))
    print('Time taken for 1 epoch: {} secs\n'.format(time.time() - start))
```

Epoch 1 Batch 0 Loss 12.3455

```
▶ def evaluate(input_document):
    input_document = document_tokenizer.texts_to_sequences([input_document])
    input_document = tf.keras.preprocessing.sequence.pad_sequences(input_document, maxlen=encoder_maxlen, padding='post', truncating='post')

    encoder_input = tf.expand_dims(input_document[0], 0)

    decoder_input = [summary_tokenizer.word_index["<go>"]]
    output = tf.expand_dims(decoder_input, 0)

    for i in range(decoder_maxlen):
```

```

def evaluate(input_document):
    input_document = document_tokenizer.texts_to_sequences([input_document])
    input_document = tf.keras.preprocessing.sequence.pad_sequences(input_document, maxlen=encoder_maxlen, padding='post', truncating='post')

    encoder_input = tf.expand_dims(input_document[0], 0)

    decoder_input = [summary_tokenizer.word_index["<go>"]]
    output = tf.expand_dims(decoder_input, 0)

    for i in range(decoder_maxlen):
        enc_padding_mask, combined_mask, dec_padding_mask = create_masks(encoder_input, output)

        predictions, attention_weights = transformer(
            encoder_input,
            output,
            False,
            enc_padding_mask,
            combined_mask,
            dec_padding_mask
        )

        predictions = predictions[:, -1:, :]
        predicted_id = tf.cast(tf.argmax(predictions, axis=-1), tf.int32)

        if predicted_id == summary_tokenizer.word_index["<stop>"]:
            return tf.squeeze(output, axis=0), attention_weights

        output = tf.concat([output, predicted_id], axis=-1)

    return tf.squeeze(output, axis=0), attention_weights

[ ] def summarize(input_document):

```

```

def summarize(input_document):
    summarized = evaluate(input_document=input_document)[0].numpy()
    summarized = np.expand_dims(summarized[1:], 0)
    return summary_tokenizer.sequences_to_texts(summarized)[0]

```

```

[ ] from huggingface_hub import notebook_login

notebook_login()

```

```

[ ] import transformers
    from datasets import load_dataset, load_metric
    import datasets
    import random
    import pandas as pd
    from IPython.display import display, HTML
    from transformers import AutoTokenizer
    from transformers import AutoModelForSeq2SeqLM, DataCollatorForSeq2Seq, Seq2SeqTrainingArguments, Seq2SeqTrainer
    import numpy as np

[ ] model_checkpoint = "Helsinki-NLP/opus-mt-sla-en"

[ ] raw_datasets = load_dataset("yhvinga/ccmatrix",
                               lang1="uk", lang2="en",
                               split='train')
    metric = load_metric("sacrebleu")

```

```
[ ] from datasets import DatasetDict
train_testvalid = raw_datasets.train_test_split(test_size=0.05)
test_valid = train_testvalid['test'].train_test_split(test_size=0.5)
raw_datasets = DatasetDict({
    'train': train_testvalid['train'],
    'test': test_valid['test'],
    'validation': test_valid['train']})
```

```
▶ tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
if "mbart" in model_checkpoint:
    tokenizer.src_lang = "uk-UK"
    tokenizer.tgt_lang = "en-XX"
```

```
[ ] if model_checkpoint in ["t5-small", "t5-base", "t5-larg", "t5-3b", "t5-11b"]:
    prefix = "translate Ukrainian to English: "
else:
    prefix = ""
```

```
▶ max_input_length = 128
max_target_length = 128
source_lang = "uk"
target_lang = "en"

def preprocess_function(examples):
    inputs = [prefix + ex[source_lang] for ex in examples["translation"]]
    targets = [ex[target_lang] for ex in examples["translation"]]
    model_inputs = tokenizer(inputs, max_length=max_input_length, truncation=True)

    # Setup the tokenizer for targets
    with tokenizer.as_target_tokenizer():
        labels = tokenizer(targets, max_length=max_target_length, truncation=True)

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

```
[ ] tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

```
[ ] from transformers import AutoModelForSeq2SeqLM, DataCollatorForSeq2Seq, Seq2SeqTrainingArguments, Seq2SeqTrainer

model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

Downloading: 100%  286M/286M [00:08<00:00, 35.5MB/s]

```
[ ] batch_size = 32
model_name = model_checkpoint.split("/")[-1]
args = Seq2SeqTrainingArguments(
    f"{model_name}-finetuned2-{source_lang}-to-{target_lang}",
    evaluation_strategy = "epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=5,
    predict_with_generate=True,
    fp16=True,
    push_to_hub=True,
```

```
[ ] data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)
```

```
[ ] def postprocess_text(preds, labels):
    preds = [pred.strip() for pred in preds]
    labels = [[label.strip()] for label in labels]

    return preds, labels

def compute_metrics(eval_preds):
    preds, labels = eval_preds
    if isinstance(preds, tuple):
        preds = preds[0]
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)

    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    decoded_preds, decoded_labels = postprocess_text(decoded_preds, decoded_labels)

    result = metric.compute(predictions=decoded_preds, references=decoded_labels)
    result = {"bleu": result["score"]}

    prediction_lens = [np.count_nonzero(pred != tokenizer.pad_token_id) for pred in preds]
    result["gen_len"] = np.mean(prediction_lens)
    result = {k: round(v, 4) for k, v in result.items()}
    return result
```

```

trainer = Seq2SeqTrainer(
    model,
    args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

```

▶ `trainer.train()`

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПЕРЕКЛАДУ ТА СУМАРИЗАЦІЇ
НОВИН**

Програма та методика тестування

КПІ.ІІ-8228.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Сергій ЛЕБІДЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Анастасія РИБАЛКО

Київ – 2022

ЗМІСТ

1 Об'єкт випробувань	3
2 Мета тестування	4
3 Методи тестування.....	5
4 Засоби та порядок тестування.....	6

					КПІ.ІП-8228.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є веб-застосунок для перекладу і сумаризації новин.

					КПІ.ІП-8228.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення у відповідно до функціональних вимог;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

					КПІ.ІП-8228.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється уся документація, яка аналізується на предмет дотримання стандартів програмування;
- динамічне тестування – застосовується в процесі виконання програми. Коректність програмного засобу перевіряється на безлічі тестів. При прогоні кожного з них збираються та аналізуються дані про проблеми в роботі програми;
- тестування «білої скриньки» – об'єктом тестування тут є внутрішня поведінка програми. Перевіряється коректність побудови всіх елементів програми та правильність їхньої взаємодії один з одним.

					КПІ.ІП-8228.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується вручну, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення так і в зручності в користуванні. Для того, щоб перевірити працездатність та відмовостійкість додатку, необхідно провести наступні тестування:

- динамічне тестування на відповідність функціональним вимогам;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування інтерфейсу користувача;
- тестування зручності використання.

					КПІ.ІП-8228.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ТЕМА ДИПЛОМНОГО ПРОЄКТУ

Графічний матеріал

КПІ.ПІ-8228.045440.05.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Сергій ЛЕБІДЬ

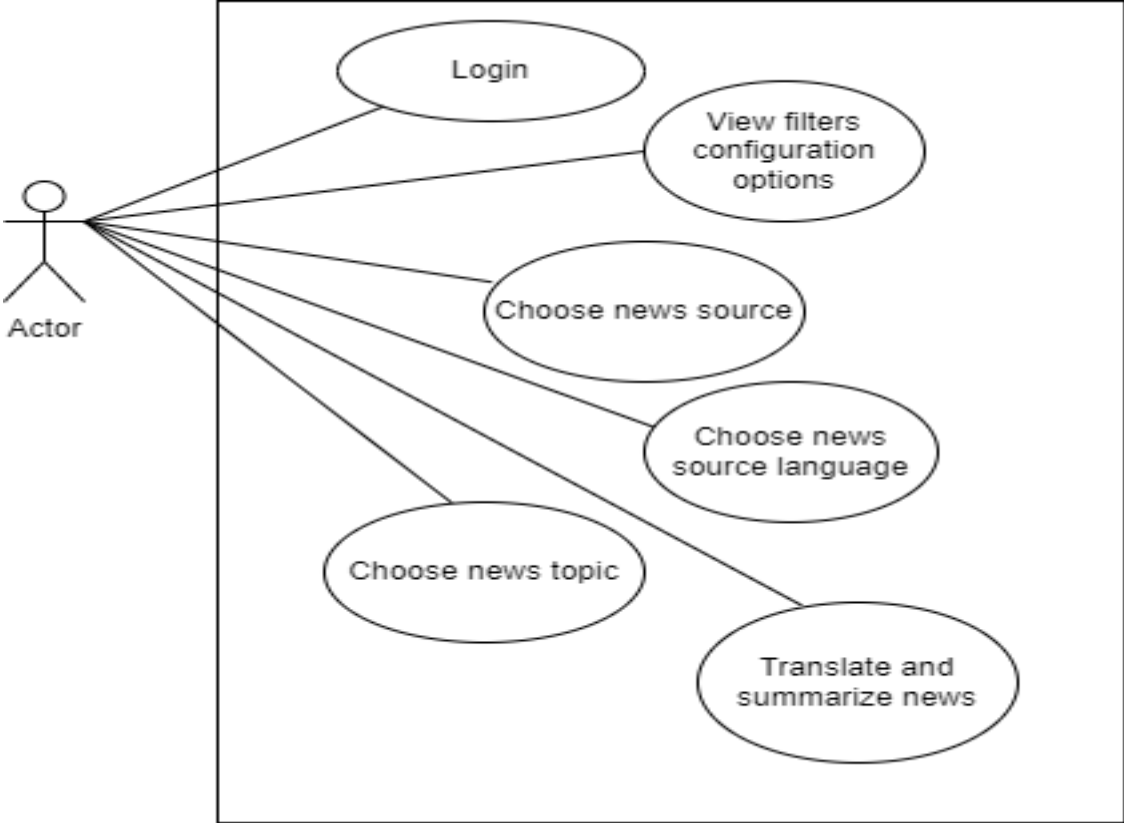
Нормоконтроль:

_____ Катерина ЛІЩУК

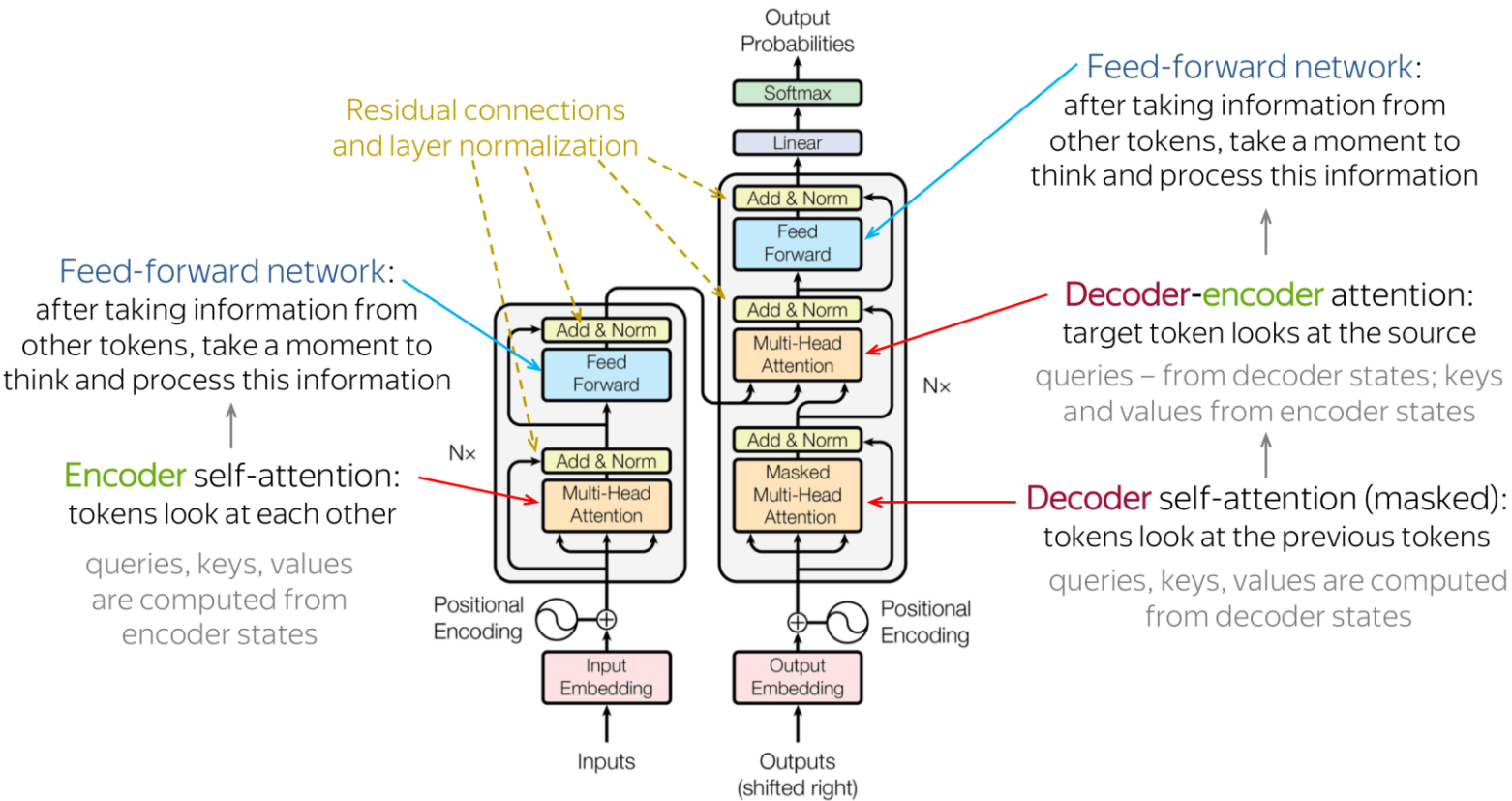
Виконавець:

_____ Анастасія РИБАЛКО

Київ – 2022



					КПІ.ІІІ-8228.045440.05.99						
Зм.	Арк.	№ докум.	Підпис	Дата	Схема структурна варіантів використань			Літ.	Арк.	Аркушів	
Розробив		Рибалко А.А.									1
Перевірів		Лебідь С.О.			Програмне забезпечення для перекладу та сумаризації новин			«КПІ імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82			
Реценз.											
Н. Контр.		Ліщук К.І.									
Затв.		Ліщук К.І.									



					КПІ.ІІІ-8228.045440.05.99			
Зм.	Арк.	№ докум.	Підпис	Дата				
Розробив	Рибалко А.А ..				Схема реалізованої архітектури трансформера	Літ.	Арк.	Аркушів
Перевірів	Лебідь С.О .						1	1
Реценз.					Програмне забезпечення для перекладу та сумаризації новин	«КПІ імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82		
Н. Контр.	Ліщук К.І.							
Затв.	Ліщук К.І.							

КПІ.ІІІ-8228.045440.05.99

weblink

https://tsn.ua/

topic

iphone

lang

ukrainian

Clear

Submit

On September 14, Apple presented new products .
Users were shown four iPhone 13 models.

					КПІ.ІІІ-8228.045440.05.99			
Зм.	Арк.	№ докум.	Підпис	Дата	Приклад роботи програмного забезпечення	Літ.	Арк.	Аркушів
Розробив	Рибалко А.А .						1	1
Перевірів	Лебідь С.О .				Програмне забезпечення для перекладу та сумаризації новин	«КПІ імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82		
Реценз.								
Н. Контр.	Ліщук К.І.							
Затв.	Ліщук К.І.Г.							